《2D 格斗游戏课程设计报告》

一、课程设计概述

1.1 设计背景与目的

随着游戏行业的迅速发展，2D 格斗游戏因其独特的玩法和视觉风格，一直深受玩家喜爱。本次课程设计旨在通过开发一款 2D 格斗游戏，深入理解和掌握 C++ 编程语言在游戏开发中的应用，包括面向对象编程、图形渲染、碰撞检测、动画处理等核心技术，同时培养逻辑思维、问题解决能力和团队协作精神。

1.2 开发环境与工具

编程语言：C++

图形库：EasyX 图形库，用于实现游戏的图形渲染和界面设计。

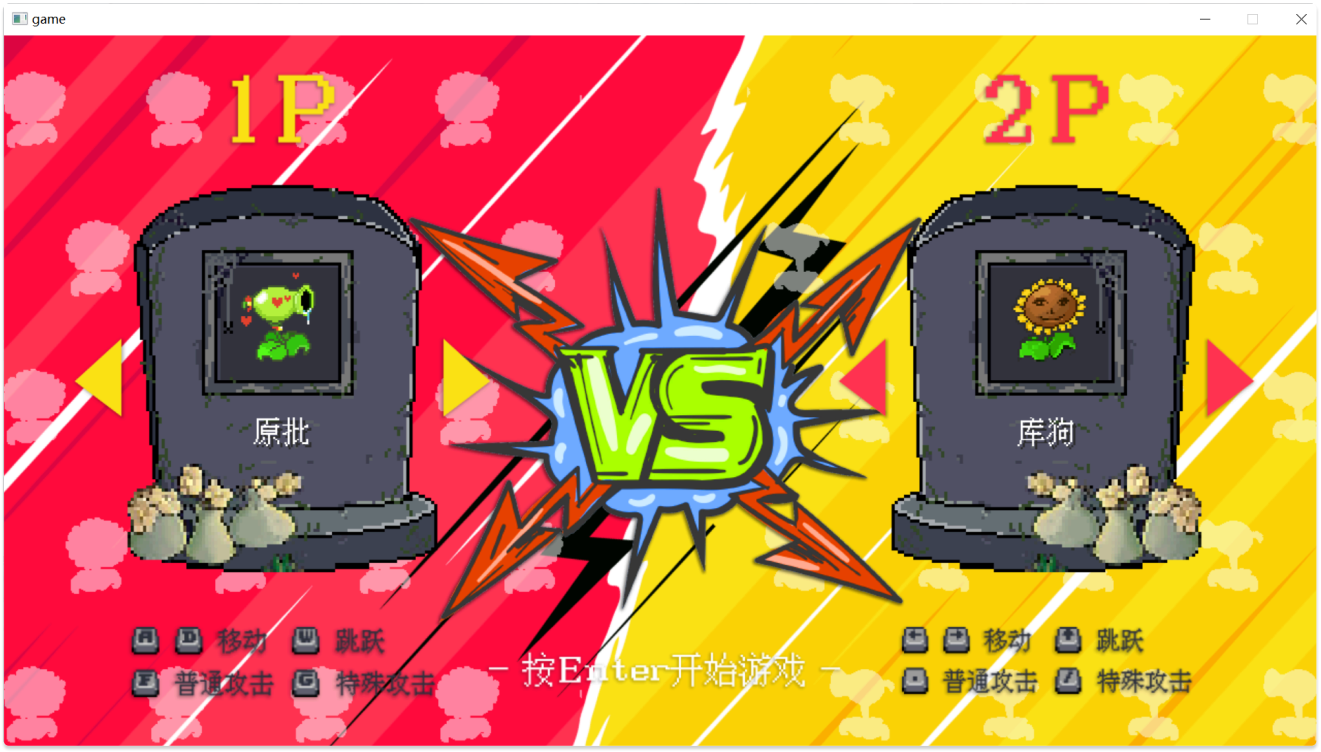
多媒体库：Windows 系统多媒体库（Winmm.lib），用于处理游戏中的音频播放。

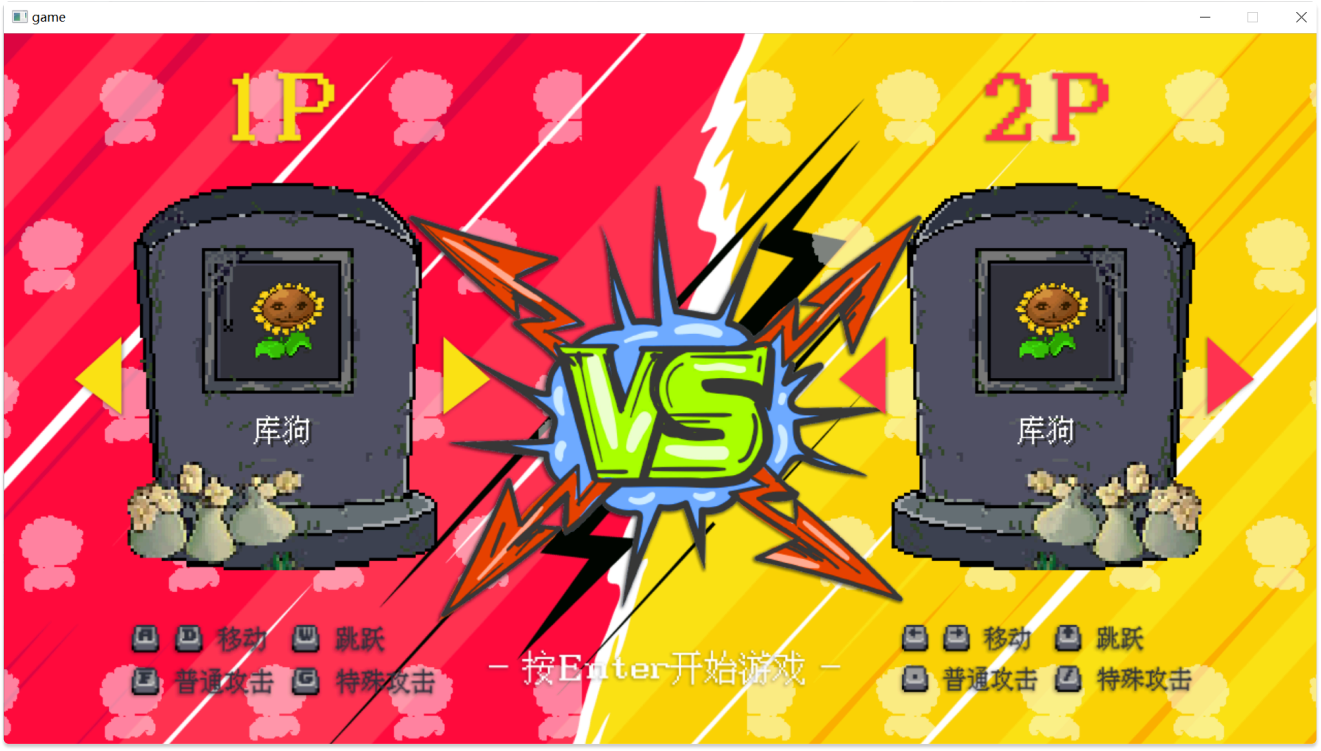
开发工具：Visual Studio，提供了强大的代码编辑、编译和调试功能。

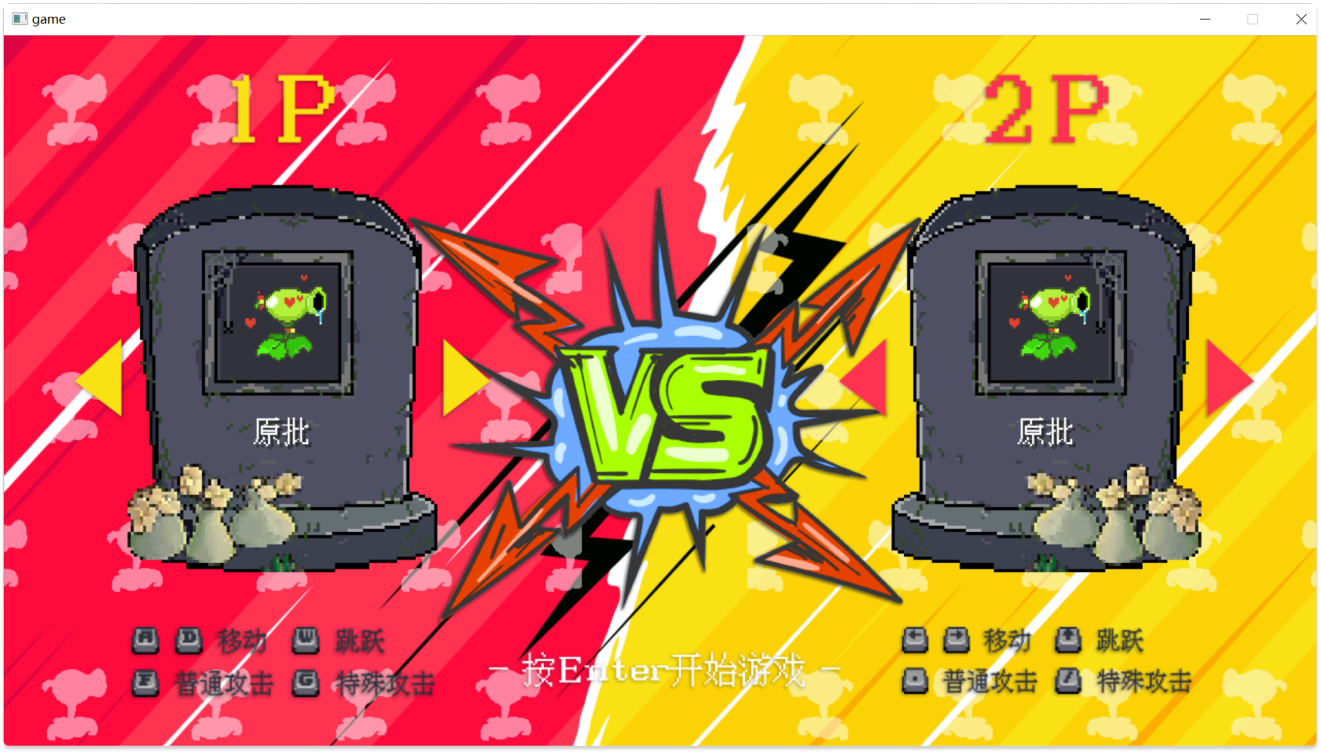
二、需求分析

2.1 功能需求

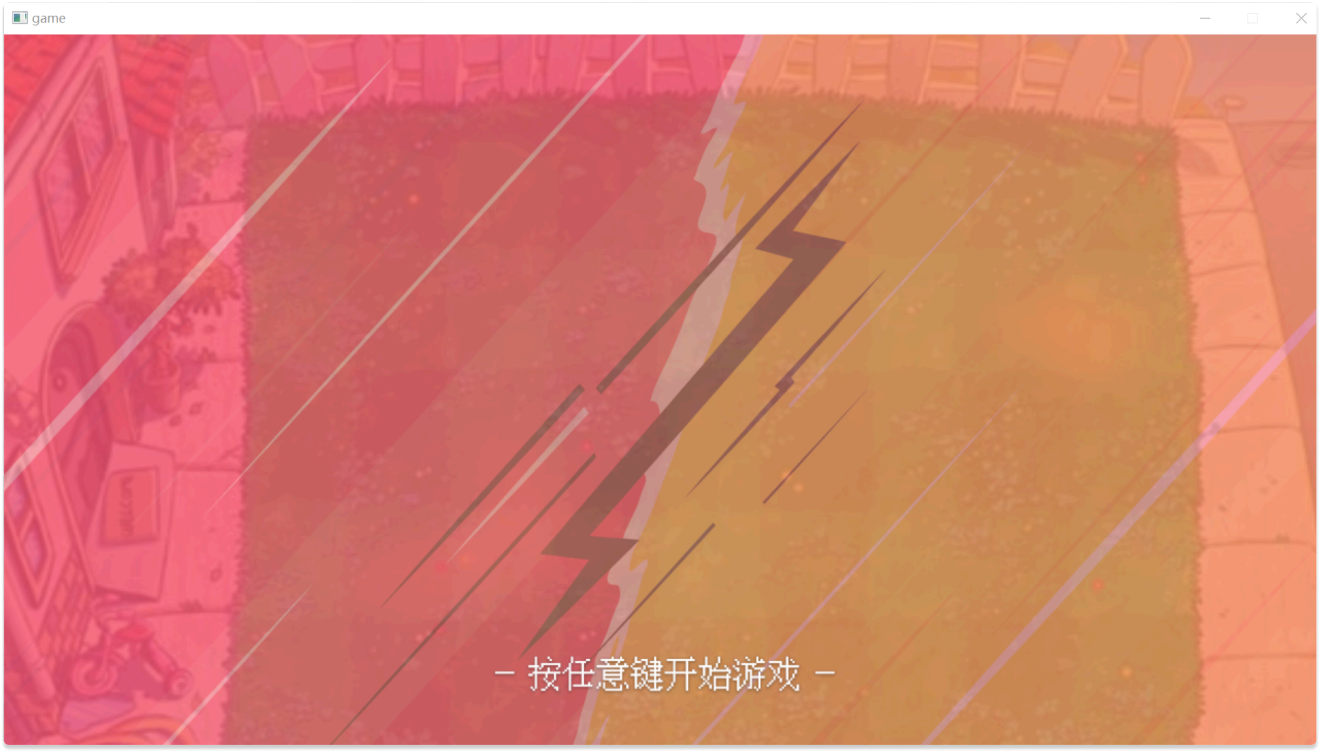
角色选择：玩家可以在选角界面选择不同的角色进行对战，如婉逗射手和龙日葵。


game
1P
2P
原批
库狗
VS
移动
跳跃
普通攻击
特殊攻击
— 按Enter开始游戏 —
移动
跳跃
普通攻击
特殊攻击


game
1P
2P
库狗
库狗
VS
移动
跳跃
普通攻击
特殊攻击
— 按Enter开始游戏 —
移动
跳跃
普通攻击
特殊攻击



场景管理 ：游戏包含主菜单场景、选角场景和游戏场景，玩家可以在不同场景之间进行切换。





角色控制 ：玩家可以通过键盘控制角色的移动、跳跃、攻击和释放技能等动作。





碰撞检测 ：实现角色与平台、角色与子弹之间的碰撞检测，确保游戏的物理效果真实可信。



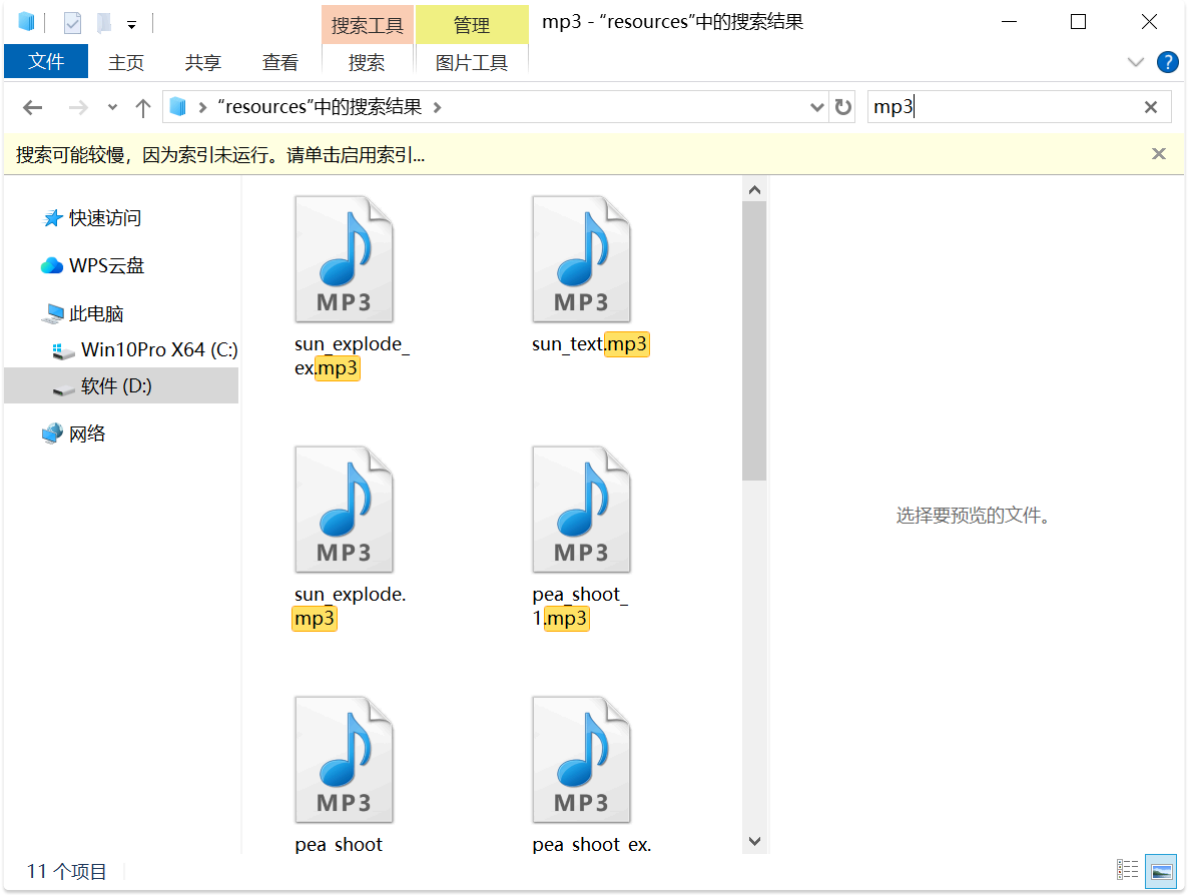


动画效果 ：为角色和技能添加动画效果，增强游戏的视觉表现力。





音效处理 ：添加游戏音效，如背景音乐、攻击音效、技能音效等，提升游戏的沉浸感。



游戏结算 ：当一方角色的生命值归零时，游戏结束并显示获胜玩家的信息。



2.2 性能需求

帧率稳定 ：游戏的帧率应保持在 60FPS 以上，确保游戏画面流畅。

响应及时 ：玩家的输入操作应能及时得到响应，避免出现明显的延迟。

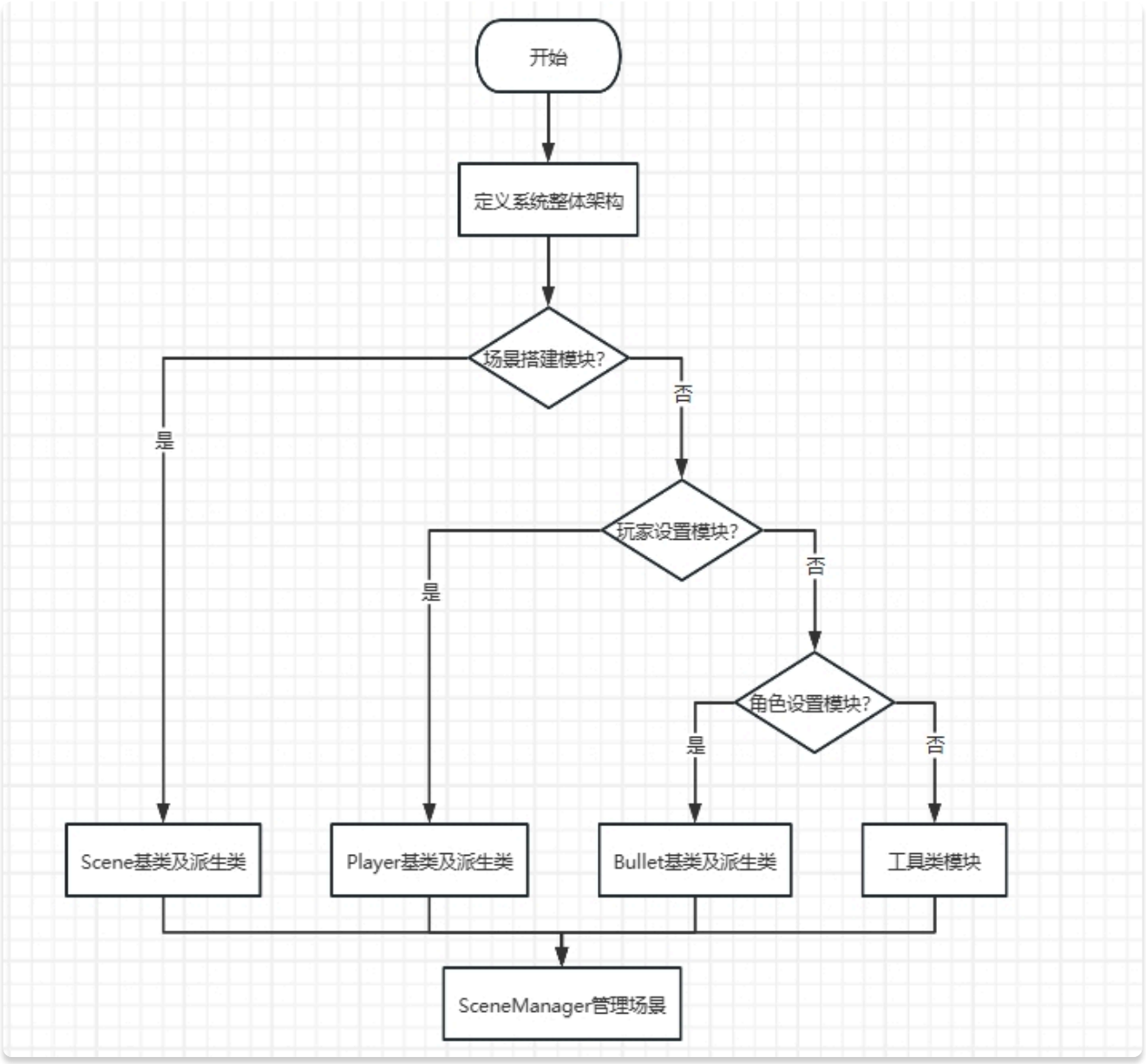
2.3 界面需求

简洁美观 ：游戏界面应简洁明了，布局合理，色彩搭配协调，给玩家带来良好的视觉体验。

易于操作 ：界面元素应易于识别和操作，方便玩家进行角色选择、开始游戏等操作。

三、系统设计

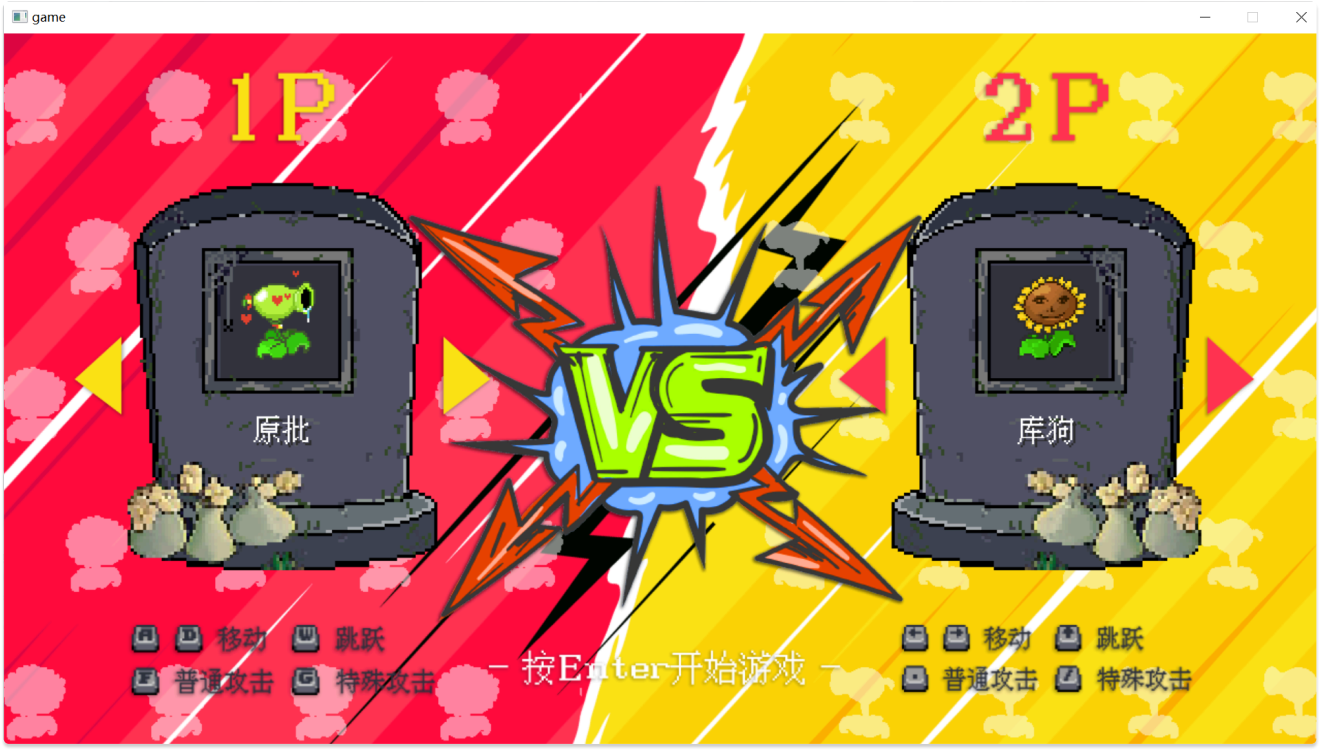
3.1 总体架构设计



游戏采用分层架构设计，主要分为以下几个模块：

场景管理模块 ：负责游戏场景的切换和管理，包括主菜单场景、选角场景和游戏场景。


game
– 按任意键开始游戏 –


game
1P
2P
原批
库狗
VS
移动
跳跃
普通攻击
特殊攻击
– 按Enter开始游戏 –
移动
跳跃
普通攻击
特殊攻击



角色管理模块 ：管理游戏中的角色，包括角色的创建、更新和绘制等操作。



子弹管理模块 ：处理游戏中的子弹，包括子弹的发射、移动、碰撞检测和销毁等操作。





平台管理模块 ：管理游戏中的平台，包括平台的创建、绘制和碰撞检测等操作。

动画管理模块 ：实现角色和技能的动画效果，包括动画的播放、暂停和重置等操作。

音效管理模块 ：处理游戏中的音效，包括背景音乐、攻击音效、技能音效等的播放和控制。

3.2 类设计

3.2.1 基础类

Vector2 类 ：表示二维向量，用于存储和处理角色、子弹等对象的位置和速度。

```
#include <iostream>
#include <cmath>
// 2D 向量类
class Vector2 {
public:
    float x = 0; // 用float的类型减少内存消耗，且运算速度更快
    float y = 0;
public:
    Vector2() = default;
    ~Vector2() = default;
```

```
    Vector2(float x, float y) : x(x), y(y) { }; // 向量设置

    Vector2 operator+(const Vector2& vec) const {
        return Vector2(x + vec.x, y + vec.y);
    }

    void operator+=(const Vector2& vec) {
        x += vec.x, y += vec.y;
    }

    void operator-=(const Vector2& vec) {
        x -= vec.x, y -= vec.y;
    }

    Vector2 operator-(const Vector2& vec) const {
        return Vector2(x - vec.x, y - vec.y);
    }

    float operator*(const Vector2& vec) const {
        return x * vec.x + y * vec.y;
    }

    Vector2 operator*(float val) const { // const 放在成员列表后，表明该成员函数不会修改对象的任何非静态数据成员
        return Vector2(x * val, y * val);
    }

    void operator*=(float val) {
        x *= val, y *= val;
    }

    // 获取向量长度
    float length() {
        return  sqrt(x * x + y * y);
    }

    //向量标准化
    Vector2 normalize() {
        float len = length();
        if (len == 0) return Vector2(0, 0);
        return Vector2(x / len, y / len); // 返回向量
    }

private:
};
```

Timer 类 ：通用计时器类，用于实现游戏中的计时功能，如技能冷却时间、无敌时间等。

```
// 通用计时器类

#include <functional>

class Timer {
public:
    Timer() = default;
```

```
    ~Timer() = default;

    void restart() { // 重置计时器状态
        pass_time = 0;
        shotted = false;
    }

    void set_wait_time(int val) { // 设置等待时间
        wait_time = val;
    }

    void set_one_shot(bool flag) { // 是否单次触发
        one_shot = flag;
    }

    void set_callback(std::function<void()> callback)    {
        this->callback = callback;
    }

    void pause() { // 暂停
        paused = true;
    }

    void resume() { // 执行
        paused = false;
    }

    void on_update(int delta) {
        if (paused) return; // 检查是否暂停，暂停就返回

        pass_time += delta; // 累加帧更新实际过去的时间
        if (pass_time ≥ wait_time) {  // 与期望的时间进行对比
            // 计时器不是单次触发 || 计时器单次触发并且没有被触发过&&回调函数存在
            if (!one_shot || (one_shot && !shotted) && callback) callback(); // 超时
则触发回调
            shotted = true; // 设置计时器的触发状态
            pass_time = 0; // 重置已经度过的时间
        }
    }

private:
    int pass_time = 0; // 已过时间
    int wait_time = 0; // 等待时间
    bool paused = false; // 是否暂停
    bool shotted = false; // 是否触发
    bool one_shot = false; // 单次触发
    std::function<void()> callback; // 触发回调
};
```

3.2.2 场景类

Scene 类：抽象基类，定义了场景的基本接口，包括进入场景、更新场景、绘制场景、处理输入和退出场景等操作。

```
#include <graphics.h>
```

```cpp
#include "camera.h"

class Scene {
public:
    Scene() = default;
    ~Scene() = default;

    // 声明虚函数
    virtual void on_enter() {} // 进入时的初始化逻辑
    virtual void on_update(int delta) {} // delta表示实际过去的时间 处理数据
    virtual void on_draw(const Camera& camera) {} // 渲染绘图
    virtual void on_input(const ExMessage& msg) {} // 玩家输入
    virtual void on_exit() {} // 退出时的卸载逻辑

private:

};
```

MenuScene 类 ：继承自 Scene 类，实现主菜单场景的功能，包括背景音乐的播放和场景切换等操作。

```cpp
// 场景菜单类
#include "scene.h"
#include "atlas.h"
#include "animation.h"
#include "camera.h"
#include "timer.h"
#include "scene_manager.h"

#include <iostream>

extern IMAGE img_menu_background;
extern Atlas atlas_peashooter_run_right;

extern SceneManager scene_manager;

class MenuScene : public Scene { // 继承Scenne基类
public:
    MenuScene() = default;
    ~MenuScene() = default;
    void on_enter() {
        mciSendString(_T("play bgm_menu repeat from 0"), NULL, 0, NULL);
    }

    void on_update(int delta) {

    }

    void on_draw(const Camera& camera) { // 渲染摄像机对象
        // 角色位置 = 世界位置 - 摄像机位置
        putimage(0, 0, &img_menu_background);

    }

    void on_input(const ExMessage& msg) {
        if (msg.message == WM_KEYUP) { // 按键抬起时(WM_KEYUP) 按下(WM_KEYDOWN)
            mciSendString(_T("play ui_confirm from 0"), NULL, 0, NULL);
```

```
            scene_manager.switch_to(SceneManager::SceneType::Selector);
        }

    }

    void on_exit() {

    }

private:
private:
    Timer timer;
    Camera camera;
    Animation animation_peashooter_run_right;

};
```

GameScene 类 **：继承自 Scene 类，实现游戏场景的功能，包括角色的移动、攻击、碰撞检测、游戏结算等操作。**

```
#include "scene.h"
#include "scene_manager.h"
#include "camera.h"
#include "bullet.h"
#include "unil.h"
#include "status_bar.h"
#include "platform.h"
#include "player.h"

extern  Player* player_1;
extern  Player* player_2;

extern IMAGE img_sky;
extern IMAGE img_hills;
extern IMAGE img_platform_large;
extern IMAGE img_platform_small;

extern IMAGE* img_player_1_avater;
extern IMAGE* img_player_2_avater;

extern IMAGE img_1P_winner;
extern IMAGE img_2P_winner;
extern IMAGE img_winner_bar;

extern Camera camera;
extern Camera main_camera;

extern SceneManager scene_manager;
extern std::vector<Platform> platform_list;

class GameScene : public Scene { // 继承Scenne基类
public:
    GameScene() = default;
    ~GameScene() = default;

    void on_enter() {
```

```
        // 由于游戏每次开始都要重置对局状态，所以把游戏结束和动效划出关掉
        is_game_over = false;
        is_slide_out_started = false;
        //  计算两层图片居中的目标位置
        // 黑幕
        pos_img_winner_bar.x = -img_winner_bar.getwidth();
        pos_img_winner_bar.y = (getheight() - img_winner_bar.getheight()) / 2;
        pos_x_img_winner_bar_dst = (getwidth() - img_winner_bar.getwidth()) / 2;
        // 文本
        pos_img_winner_text.x = pos_img_winner_bar.x;
        pos_img_winner_text.y = (getheight() - img_1P_winner.getheight()) / 2;
        pos_x_img_winner_text_dst = (getwidth() - img_1P_winner.getwidth()) / 2;

        timer_winner_slide_in.restart();
        timer_winner_slide_in.set_wait_time(2500);
        timer_winner_slide_in.set_one_shot(true);
        timer_winner_slide_in.set_callback([&]() {
            is_slide_out_started = true; // 回调为结算动画划出
            });

        timer_winner_slide_out.restart();
        timer_winner_slide_out.set_wait_time(1000);
        timer_winner_slide_out.set_one_shot(true);
        timer_winner_slide_out.set_callback([&]() {
            scene_manager.switch_to(SceneManager::SceneType::Selector); // 划出后回调
回主菜单界面
            });



        // 设置图片
        status_bar_1P.set_avater(img_player_1_avater);
        status_bar_2P.set_avater(img_player_2_avater);
        // 设置位置
        status_bar_1P.set_position(235, 625);
        status_bar_2P.set_position(675, 625);

        // 为角色赋初值
        player_1->set_position(200, 50);
        player_2->set_position(975, 50);

        // 居中绘制背景
        pos_img_sky.x = (getwidth() - img_sky.getwidth()) / 2;
        pos_img_sky.y = (getheight() - img_sky.getheight()) / 2;

        pos_img_hills.x = (getwidth() - img_hills.getwidth()) / 2;
        pos_img_hills.y = (getheight() - img_hills.getheight()) / 2;

        platform_list.resize(4);
        Platform& large_platform = platform_list[0];
        large_platform.img = &img_platform_large; // 列表第一位存储大平台
        // 大平台的渲染位置左
        large_platform.render_positon.x = 122; // 左边界
        large_platform.render_positon.y = 455; // 高度
         // 碰撞外形坐标
        large_platform.shape.left = (float)large_platform.render_positon.x + 30; //
平台左边界(平台图像向右 +30)
```

```
        large_platform.shape.right = (float)large_platform.render_positon.x +
img_platform_large.getwidth() - 30; // 右边界(向左 + 画布的宽 -30)
        large_platform.shape.y = (float)large_platform.render_positon.y + 60; // 平
台的高

        Platform& small_platform_1 = platform_list[1];
        small_platform_1.img = &img_platform_small;
        small_platform_1.render_positon.x = 175;
        small_platform_1.render_positon.y = 360;
        small_platform_1.shape.left = (float)small_platform_1.render_positon.x + 40;
        small_platform_1.shape.right = (float)small_platform_1.render_positon.x +
img_platform_small.getwidth() - 40;
        small_platform_1.shape.y = (float)small_platform_1.render_positon.y +
img_platform_small.getheight() / 2;

        Platform& small_platform_2 = platform_list[2];
        small_platform_2.img = &img_platform_small;
        small_platform_2.render_positon.x = 855;
        small_platform_2.render_positon.y = 360;
        small_platform_2.shape.left = (float)small_platform_2.render_positon.x + 40;
        small_platform_2.shape.right = (float)small_platform_2.render_positon.x +
img_platform_small.getwidth() - 40;
        small_platform_2.shape.y = (float)small_platform_2.render_positon.y +
img_platform_small.getheight() / 2;

        Platform& small_platform_3 = platform_list[3];
        small_platform_3.img = &img_platform_small;
        small_platform_3.render_positon.x = 515;
        small_platform_3.render_positon.y = 225;
        small_platform_3.shape.left = (float)small_platform_3.render_positon.x + 40;
        small_platform_3.shape.right = (float)small_platform_3.render_positon.x +
img_platform_small.getwidth() - 40;
        small_platform_3.shape.y = (float)small_platform_3.render_positon.y +
img_platform_small.getheight() / 2;

        mciSendString(_T("play bgm_game repeat from 0"), NULL, 0, NULL); // 播放战斗
音乐

    }

    void on_update(int delta) {
        player_1->on_update(delta);
        player_2->on_update(delta);

        main_camera.on_update(delta); // 更新主摄像机对象

        // 子弹移除逻辑
        bullet_list.erase(std::remove_if(
            bullet_list.begin(), bullet_list.end(),
            [](const Bullet* bullet) {
                bool detetable = bullet->check_can_remove(); // 如果子弹可以移除就返回
true
                if (detetable) delete bullet; // 释放子弹内存
                return detetable;
            }),
            bullet_list.end());
```

```
        for (Bullet* bullet : bullet_list) bullet->on_update(delta); // 遍历每一个子弹对象(更新子弹)

        // 获得玩家的位置
        const Vector2& position_player_1 = player_1->get_position();
        const Vector2& position_player_2 = player_2->get_position();
        // 如果玩家掉出世界
        if (position_player_1.y ≥ getheight()) player_1->set_hp(0); //  坐标超出窗口下方，生命值为0
        if (position_player_2.y ≥ getheight()) player_2->set_hp(0);

        if (player_1->get_hp() ≤ 0 || player_2->get_hp() ≤ 0) { // 生命归0
            if (!is_game_over) { // 游戏结束的最后一帧结束背景音乐，播放win音乐
                mciSendString(_T("stop bgm_game"), NULL, 0, NULL);
                mciSendString(_T("play ui_win from 0"), NULL, 0, NULL);
            }
            is_game_over = true; // 游戏结束
        }

        // 获得玩家的状态
        status_bar_1P.set_hp(player_1->get_hp());
        status_bar_1P.set_mp(player_1->get_mp());
        status_bar_2P.set_hp(player_2->get_hp());
        status_bar_2P.set_mp(player_2->get_mp());

        if (is_game_over) { // 如果游戏结束
            // 累加结算标题两层图片的水平位置
            pos_img_winner_bar.x += (int)(speed_winner_bar * delta);
            pos_img_winner_text.x += (int)(speed_winner_text * delta);
            // 根据是否划出执行不同指令
            if (!is_slide_out_started) // 如果没有执行划出逻辑，说明正在进行停顿展示
            {
                timer_winner_slide_in.on_update(delta);
                // 更新逻辑不让图片超过最大展示位置
                if (pos_img_winner_bar.x > pos_x_img_winner_bar_dst) pos_img_winner_bar.x = pos_x_img_winner_bar_dst;
                if (pos_img_winner_text.x > pos_x_img_winner_text_dst) pos_img_winner_text.x = pos_x_img_winner_text_dst;
            }
            else timer_winner_slide_out.on_update(delta);

        }
    }

    void on_draw(const Camera& camera) {
        putimage_alpha(camera, pos_img_sky.x, pos_img_sky.y, &img_sky);
        putimage_alpha(camera, pos_img_hills.x, pos_img_hills.y, &img_hills);

        for (const auto& platform : platform_list) platform.on_draw(camera); // 通过迭代遍历平台列表

        if (is_debug) {
            settextcolor(RGB(255, 0, 0));
            outtextxy(15, 15, _T("已开启调试，Q键关闭"));
        }

        player_1->on_draw(camera);
        player_2->on_draw(camera);
```

```
        for (const Bullet* bullet : bullet_list) bullet->on_draw(camera); // 按帧绘制

        if (is_game_over) { // 游戏结束绘制黑幕和文本
            putimage_alpha(pos_img_winner_bar.x, pos_img_winner_bar.y,
&img_winner_bar);
            putimage_alpha(pos_img_winner_text.x, pos_img_winner_text.y, player_1-
>get_hp() > 0 ? &img_1P_winner : &img_2P_winner); // 绘制胜利者文本
        }else {
            // 绘制状态条长度
            status_bar_1P.on_draw();
            status_bar_2P.on_draw();
        }

    }

    void on_input(const ExMessage& msg) {
        player_1->on_input(msg);
        player_2->on_input(msg);

        switch (msg.message) {
        case WM_KEYUP:
            //'Q'
            if (msg.vkcode == 0x51) is_debug ^= 1;
            break;
        default:
            break;
        }

    }

    void on_exit() {
        delete player_1;
        player_1 = nullptr;
        delete player_2;
        player_2 = nullptr;

        is_debug = false; // 退出禁用debug

        // 重置相机对象 清空子弹数组
        bullet_list.clear();
        main_camera.reset();
    }

private:
    const float speed_winner_bar = 3.0f; // 结算动效背景移动速度
    const float speed_winner_text = 2.5f; // 结算动效文本移动速度
private:
    POINT pos_img_sky = { 0 }; // 天空背景
    POINT pos_img_hills = { 0 }; // 山脉背景

    StatusBar status_bar_1P; // 玩家1状态条
    StatusBar status_bar_2P; // 玩家2状态条

    bool is_game_over = false; // 游戏结束

    POINT pos_img_winner_bar = { 0 }; // 结算动效背景位置
    POINT pos_img_winner_text = { 0 }; // 结算动效文本位置
```

```
    int pos_x_img_winner_bar_dst = 0; // 结算动效背景移动的目标位置
    int pos_x_img_winner_text_dst = 0; // 结算动效文本移动的目标位置
    Timer timer_winner_slide_in; // 结算动效划入计时器
    Timer timer_winner_slide_out; // 结算动效滑出计时器
    bool is_slide_out_started = false; // 结算动效是否开始划出
}
```

3.2.3 角色类

Player 类：玩家基类，定义了角色的基本接口，包括角色的更新、绘制、输入处理、攻击和技能释放等操作。

```
#pragma once

#include "camera.h"
#include "vector2.h"
#include "animation.h"
#include "player_id.h"
#include "platform.h"
#include "player_id.h"
#include "bullet.h"
#include "unil.h"
#include "particle.h"

#include <graphics.h>

extern bool is_debug;

extern IMAGE img_1P_cursor;
extern IMAGE img_2P_cursor;

extern Atlas atlas_run_effect;
extern Atlas atlas_jump_effect;
extern Atlas atlas_land_effect;

extern std::vector<Bullet*> bullet_list;
extern std::vector<Platform> platform_list; // 平台列表

class Player {
public:
    Player(bool facing_right = true) : is_facing_right(facing_right) {
        current_animation = &animation_idle_right; // 设置指向向右静止图像的指针
        was_w_key_down = false;
        was_up_key_down = false;

        timer_attack_cd.set_wait_time(attack_cd); // 设置cd时间
        timer_attack_cd.set_one_shot(true); // 设置为单次触发
        timer_attack_cd.set_callback([&]() {
            can_attack = true;
        });

        // 初始话无敌时间计时器
        timer_invulnerable.set_wait_time(750); // 无敌时间
        timer_invulnerable.set_one_shot(true); // 单次触发
        timer_invulnerable.set_callback([&]() {
            is_invulnerable = false; // 回调成不是无敌
```

```
        });
        // 初始化无敌时间闪烁计时器
        timer_invulnerable_blink.set_wait_time(75);
        timer_invulnerable_blink.set_callback([&]() {
            is_showing_sketch_frame ^= 1; // 切换回调状态(反转当前剪影的布尔变量)
        });

        // 奔跑计时器
        timer_run_effect_generation.set_wait_time(75);
        timer_run_effect_generation.set_callback([&](){
            Vector2 particle_position;
            IMAGE* frame = atlas_run_effect.get_image(0); // 回调到第0索引的图像
            // 计算特效渲染(左上角)生成位置
            particle_position.x = position.x + (size.x - frame->getwidth()) / 2;
            particle_position.y = position.y + size.y - frame->getheight();
            particle_list.emplace_back(particle_position, &atlas_run_effect, 45); //
向粒子对象列表中存入新的生成坐标，和图集
        });

        // 死亡计时器
        timer_die_effect_generation.set_wait_time(35);
        timer_die_effect_generation.set_callback([&]() {
            Vector2 particle_position;
            IMAGE* frame = atlas_run_effect.get_image(0); // 回调到第0索引的图像
            // 计算特效渲染(左上角)生成位置
            particle_position.x = position.x + (size.x - frame->getwidth()) / 2;
            particle_position.y = position.y + size.y - frame->getheight();
            particle_list.emplace_back(particle_position, &atlas_run_effect, 150);
// 向粒子对象列表中存入新的生成坐标，和图集
        });

        // 跳跃特效
        animation_jump_effect.set_atlas(&atlas_jump_effect);
        animation_jump_effect.set_interval(25);
        animation_jump_effect.set_loop(false);
        animation_jump_effect.set_callback([&]()
            {
                is_jump_effect_visible = false;
            });
        // 落地特效
        animation_land_effect.set_atlas(&atlas_land_effect);
        animation_land_effect.set_interval(50);
        animation_land_effect.set_loop(false);
        animation_land_effect.set_callback([&]()
            {
                is_land_effect_visible = false;
            });

        // 光标显示计时器
        timer_cursor_visibility.set_wait_time(30000);
        timer_cursor_visibility.set_one_shot(true); //  单次触发
        timer_cursor_visibility.set_callback([&]() {
            is_cursor_visible = false; // 回调成不显示光标
            });

        hp = 200;
        mp = 0;
    }
```

```
    ~Player() = default;

    virtual void on_update(int delta) {
        int direction = is_right_key_down - is_left_key_down; // 通过做差的方式判断移动
方向

        if (direction != 0) {
            if(!is_attacking_ex) is_facing_right = direction > 0; // 角色开大不能转向
            current_animation = is_facing_right ? &animation_run_right :
&animation_run_left;
            float distance = direction * run_velocity * delta; // 向量 * 速度 * 时间
            on_run(distance);
        }else {
            current_animation = is_facing_right ? &animation_idle_right :
&animation_idle_left;
            timer_run_effect_generation.pause(); // 停止跑动特效生成
        }
        // 开大变化成大招动画
        if (is_attacking_ex) current_animation = is_facing_right ?
&animation_attack_ex_right : &animation_attack_ex_left;
        // 死亡的时候
        if (hp <= 0) current_animation = last_hurt_direction.x < 0 ?
&animation_die_left : &animation_die_right; // 根据受击方向觉得角色飞行方向

        current_animation->on_update(delta);
        animation_jump_effect.on_update(delta);
        animation_land_effect.on_update(delta);

        timer_attack_cd.on_update(delta); // 更新计时器
        timer_invulnerable.on_update(delta);
        timer_invulnerable_blink.on_update(delta);
        timer_run_effect_generation.on_update(delta); // 添加定时器对象
        timer_cursor_visibility.on_update(delta);

        if (hp <= 0) timer_die_effect_generation.on_update(delta); // 没血了就添加死亡
计时器对象

        particle_list.erase(std::remove_if( // 清除用过的特效
            particle_list.begin(), particle_list.end(),
            [](const Particle& particle)
            {
                return !particle.check_valid(); // 如果粒子无效了就取反变成true执行删除逻
辑
            }),
            particle_list.end());
        for (Particle& particle : particle_list) particle.on_update(delta); // 遍历执
行特效

        if (is_showing_sketch_frame) sketch_image(current_animation->get_frame(),
&img_sketch); // 获取当前帧图片


        //// 处理组合按键，只在按下 W 键的瞬间触发跳跃
        //if (id == PlayerID::P1) {
        //  if (is_w_key_down && !was_w_key_down) {
        //      if (is_right_key_down) {
        //          on_jump();
        //      }
```

```
        //      else if (is_left_key_down) {
        //          on_jump();
        //      }
        //      else {
        //          on_jump();
        //      }
        //  }
        //}
        //was_w_key_down = is_w_key_down; // 更新 W 键上一帧的状态

        //if (id == PlayerID::P2) {
        //  if (is_up_key_down && !was_up_key_down) { // 按下且上一帧不是按下状态
        //      if (is_right_key_down) {
        //          on_jump();
        //      }
        //      else if (is_left_key_down) {
        //          on_jump();
        //      }
        //      else {
        //          on_jump();
        //      }
        //  }
        //}
        //was_up_key_down = is_up_key_down; // 更新 W 键上一帧的状态

        move_and_collide(delta); // 物理场景设置 + 攻击
    }

    virtual void on_run(float distance) {
        if (is_attacking_ex) return; // 如果正在释放技能就不得移动
        position.x += distance; // 修改水平位置
        timer_run_effect_generation.resume(); // 恢复跑动特效计时器
    }

    virtual void on_land() {
        is_land_effect_visible = true;
        animation_land_effect.reset();

        IMAGE* effect_frame = animation_land_effect.get_frame();
        position_land_effect.x = position.x + (size.x - effect_frame->getwidth()) /
2;
        position_land_effect.y = position.y + size.y - effect_frame->getheight();
    }

    virtual void on_jump() {

        if (jump_count >= MAX_JUMPS) {
            jump_count = 0;
            return;
        }
        if ((jump_count == 0 && velocity.y != 0) || is_attacking_ex) return;    //
如果在空中或者开大了就不能跳了
        velocity.y += jump_velocity;
        ++jump_count;

        is_jump_effect_visible = true; // 可见
        animation_jump_effect.reset(); // 重置动画状态
```

```
        // 计算得到跳跃特效位置
        IMAGE* effect_frame = animation_jump_effect.get_frame();
        position_jump_effect.x = position.x + (size.x - effect_frame->getwidth()) /
2;
        position_jump_effect.y = position.y + size.y - effect_frame->getheight();
    }

    virtual void on_draw(const Camera& camera) {
        if (is_jump_effect_visible) animation_jump_effect.on_draw(camera,
(int)position_jump_effect.x, (int)position_jump_effect.y);
        if (is_land_effect_visible) animation_land_effect.on_draw(camera,
(int)position_land_effect.x, (int)position_land_effect.y);

        for (const Particle& particle : particle_list) particle.on_draw(camera); //
遍历绘制特效

        if (hp > 0 && is_invulnerable && is_showing_sketch_frame)
putimage_alpha(camera, (int)position.x, (int)position.y, &img_sketch);
        else current_animation->on_draw(camera, (int)position.x, (int)position.y);

        if (is_cursor_visible) { //  光标可见
            switch (id)
            {
            case PlayerID::P1: // 角色光标跟随摄像机渲染在角色头顶
                putimage_alpha(camera, (int)(position.x + (size.x -
img_1P_cursor.getwidth()) / 2), (int)(position.y - img_1P_cursor.getheight()),
&img_1P_cursor);
                break;
            case PlayerID::P2:
                putimage_alpha(camera, (int)(position.x + (size.x -
img_2P_cursor.getwidth()) / 2), (int)(position.y - img_2P_cursor.getheight()),
&img_2P_cursor);
                break;
            }
        }

        if (is_debug) { //  开启 debug 绘制轮廓线
            setlinecolor(RGB(0, 125, 255));
            rectangle((int)position.x, (int)position.y, (int)(position.x + size.x),
(int)(position.y + size.y)); // 在窗口坐标下绘制线框
        }

    }

    virtual void on_input(const ExMessage& msg) {
        switch (msg.message)
        {
        case WM_KEYDOWN: // 按下把方向左右执行
            switch (id)
            {
            case PlayerID::P1:
                switch (msg.vkcode)
                {
                    //'A'
                case 0x41:
                    is_left_key_down = true;
                    break;
                    // 'D'
```

```
            case 0x44:
                is_right_key_down = true;
                break;
                // 'W'
            case 0x57:
                //is_w_key_down = true;
                on_jump();
                break;
                //'F'
            case 0x46:
                if (can_attack) {
                    on_attack();
                    can_attack = false; // 设置为不可以放普通攻击
                    timer_attack_cd.restart(); // 重置cd
                }
                break;
            // 'G'
            case 0x47:
                if (mp ≥ 100) { // 如果满怒气条了
                    on_attack_ex();
                    mp = 0;
                }
            }
            break;
        case PlayerID::P2:
            switch (msg.vkcode)
            {
                //'←'
            case VK_LEFT:
                is_left_key_down = true;
                break;
                // '→'
            case VK_RIGHT:
                is_right_key_down = true;
                break;

            case VK_UP:
                //is_up_key_down = true;
                on_jump();
                break;
                // '.'
            case VK_OEM_PERIOD:
                if (can_attack) {
                    on_attack();
                    can_attack = false; // 设置为不可以放普通攻击
                    timer_attack_cd.restart(); // 重置cd
                }
                break;
                //'/'
            case VK_OEM_2:
                if (mp ≥ 100) {
                    on_attack_ex();
                    mp = 0;
                }
            }
        }
        break;
    case WM_KEYUP: // 抬起关闭左右变量
```

```
            switch (id)
            {
            case PlayerID::P1:
                switch (msg.vkcode)
                {
                    //'A'
                case 0x41:
                    is_left_key_down = false;
                    break;
                    // 'D'
                case 0x44:
                    is_right_key_down = false;
                    break;
                //case 0x57:
                //  is_w_key_down = false;
                //  break;
                }
                break;

            case PlayerID::P2:
                switch (msg.vkcode)
                {
                    //'A'
                case VK_LEFT:
                    is_left_key_down = false;
                    break;
                    // 'D'
                case VK_RIGHT:
                    is_right_key_down = false;
                    break;
                //case VK_UP:
                //  is_up_key_down = false;
                //  break;
                }
                break;
            }
            break;

        }
    }

    void set_id(PlayerID id) { // 对玩家id进行赋值
        this->id = id;
    }

    void set_position(float x, float y) {
        position.x = x, position.y = y;
    }

    const Vector2& get_position() const { // 获取敌方角色位置·
        return position;
    }

    const Vector2& get_size() const { // 获取敌方角色尺寸
        return size;
    }

    void set_hp(int val) { // 设置玩家生命值
```

```
        hp = val;
    }

    int get_hp() const{ // 获取生命
        return hp;
    }

    int get_mp() const { // 获取能量
        return mp;
    }

    virtual void on_attack() { // 平a

    }

    virtual void on_attack_ex() { // 大招

    }

    // 无敌状态设置
    void make_invulnerable() {
        is_invulnerable = true; // 开启无敌
        timer_invulnerable.restart(); // 重置无敌定时器
    }

protected:

    Animation* current_animation = nullptr; // 指向正在播放的动画

    PlayerID id = PlayerID::P1; // 玩家id

    bool is_left_key_down = false; // 向左的按键是否按下
    bool is_right_key_down = false; // 向右的按键是否按下
    bool is_w_key_down = false; // 向上的按键是否按下
    bool is_up_key_down = false; // 向上的按键是否按下
    bool was_w_key_down = false; // 新增变量记录 W 键上一帧的状态
    bool was_up_key_down = false; // 新增变量记录 up 键上一帧的状态

    bool is_facing_right = true; // 角色是否朝向右

    int attack_cd = 500; // 平a冷却时间
    bool can_attack = true; // 可以释放普通攻击
    Timer timer_attack_cd; // 平a冷却时间定时器

    bool is_attacking_ex = false; // 是否在开大招

    IMAGE img_sketch; // 动画帧剪影图片
    bool is_invulnerable = false; // 角色是否处于无敌状态
    bool is_showing_sketch_frame = false; // 当前帧是否应该显示剪影
    Timer timer_invulnerable; // 无敌状态定时器
    Timer timer_invulnerable_blink; // 无敌状态闪烁定时器

    std::vector<Particle> particle_list; // 粒子对象列表

    Timer timer_run_effect_generation; // 跑动特效粒子发射计时器
    Timer timer_die_effect_generation; // 死亡特效粒子发射计时器

    bool is_cursor_visible = true; // 玩家的光标指示器是否显示
```

```
    Timer timer_cursor_visibility; // 玩家光标指示器可见性定时器

    Vector2 last_hurt_direction; // 最后一次受击方向

    bool will_jump = 0; // 预输入跳跃标志

protected:
        const float run_velocity = 0.55f; // 跑动速度
        const float gravity = 1.6e-3f; // 玩家重力
        const float jump_velocity = -0.60f; // 跳跃速度
        int jump_count = 0;
        const int MAX_JUMPS = 2;

protected:
        int mp = 0; // 角色能量
        int hp = 0; // 角色血量

        Vector2 size; // 角色尺寸
        Vector2 position; // 角色位置(x:水平 y:数值)
        Vector2 velocity; // 记录玩家速度

        Animation animation_idle_left; // 向左静止图画
        Animation animation_idle_right; // 向右静止图画
        Animation animation_run_left;  // 向左奔跑图画
        Animation animation_run_right; // 向右奔跑图画
        Animation animation_attack_ex_left; // 向左攻击的特殊动画
        Animation animation_attack_ex_right; // 向右攻击的特殊动画
        Animation animation_jump_effect; // 跳跃特效动画
        Animation animation_land_effect; // 落地特效动画
        Animation animation_die_left; // 角色向左死亡动画
        Animation animation_die_right; // 角色向右死亡动画


        bool is_jump_effect_visible = false; // 跳跃特效是否可见
        bool is_land_effect_visible = false; // 落地特效是否可见

        Vector2 position_jump_effect; // 跳跃动画播放位置
        Vector2 position_land_effect; // 落地动画播放位置

    protected:
        // 物理逻辑
        void move_and_collide(int delta) {

            float last_velocity_y = velocity.y; // 记录上一帧的速度
             // 设置重力
            velocity.y += gravity * delta; // 根据重力加速度的值对玩家的速度进行累加
            position += velocity * (float)delta; // 根据速度值对玩家的位置更新

            if (hp <= 0) return; // 防止角色在死亡过程中与场景中的障碍物发生判断

            if (velocity.y > 0) {
                for (const auto& platform : platform_list) {
                    const Platform::CollisionShape& shape = platform.shape; // 平台的
碰撞外形
                    // 图形是否在水平方向上重合只需要去二者最右边的值和二者最左边的值进行做
差，如果结果小于两个宽度之和，说明这两个图形在水平方向上有重合部分
                    bool is_collde_x = (max(position.x + size.x, shape.right) -
min(position.x, shape.left) <= size.x + (shape.right - shape.left));
```

```
                    bool is_collde_y = (shape.y ≥ position.y && shape.y ≤
position.y + size.y);

                    if (is_collde_x && is_collde_y) { // 水平方向和竖直方向都发生碰撞
                        float delta_pos_y = velocity.y * delta; // 这一帧角色在数值方向
上移动的距离
                        float last_tick_foot_pos_y = position.y + size.y -
delta_pos_y; // 计算上一帧玩家脚底的位置 角色位置 + 角色尺寸 - 这一帧的距离
                        if (last_tick_foot_pos_y ≤ shape.y) { // 前一帧玩家是否整个穿
过了平台
                            position.y = shape.y - size.y;
                            velocity.y = 0; // 重置y方向速度

                            if (last_velocity_y ≠ 0) on_land(); // 与平台碰撞并且上一
帧速度不是0的时候，触发落地特效

                            break;
                        }
                    }
                }
            }

            if(!is_invulnerable) { // 如果不处于无敌状态
                for (Bullet* bullet : bullet_list) {
                    if (!bullet→get_valid() || bullet→get_collide_target() ≠ id)
continue; // 当前子弹不可以发生碰撞 碰撞目标不能是发射者自己

                    if (bullet→check_collision(position, size)) { // 如果可以发生碰撞
                        make_invulnerable(); // 如果进入就进入无敌状态
                        bullet→on_collide(); // 碰撞就调用回调函数
                        bullet→set_valid(false); // 设置这个子弹不可以继续发生碰撞
                        hp -= bullet→get_damge(); // 减去这个子弹所造成的伤害

                        // 子弹位置与玩家位置进行做差，更新玩家最后一次受击方向
                        last_hurt_direction = bullet→get_positsion() - position;
                        if (hp ≤ 0) { // 没血了设置尸体起飞水平速度和数值速度
                            velocity.x = last_hurt_direction.x < 0 ? 0.35f : -0.35f;
                            velocity.y = -1.0f;
                        }
                    }
                }
            }

        }
};
```

PeashooterPlayer 类 **：继承自 Player 类，实现婉逗射手角色的功能，包括普攻、大招的释放和动画效果等操作。**

```
#pragma once

#include "player.h"
#include "pea_bullet.h"
#include "bullet.h"

#include <vector>
#include <iostream>
```

```cpp
extern Atlas atlas_peashooter_idle_left;
extern Atlas atlas_peashooter_idle_right;
extern Atlas atlas_peashooter_run_left;
extern Atlas atlas_peashooter_run_right;
extern Atlas atlas_peashooter_attack_ex_left;
extern Atlas atlas_peashooter_attack_ex_right;
extern Atlas atlas_peashooter_die_left;
extern Atlas atlas_peashooter_die_right;

extern Camera main_camera;

class PeashooterPlayer : public Player {
public:
    PeashooterPlayer(bool facing_right = true) : Player(facing_right) { // 子类朝向传递给父类
        // 获取动画集
        animation_idle_left.set_atlas(&atlas_peashooter_idle_left);
        animation_idle_right.set_atlas(&atlas_peashooter_idle_right);
        animation_run_left.set_atlas(&atlas_peashooter_run_left);
        animation_run_right.set_atlas(&atlas_peashooter_run_right);
        animation_attack_ex_left.set_atlas(&atlas_peashooter_attack_ex_left);
        animation_attack_ex_right.set_atlas(&atlas_peashooter_attack_ex_right);
        animation_die_left.set_atlas(&atlas_peashooter_die_left);
        animation_die_right.set_atlas(&atlas_peashooter_die_right);

        // 设置帧间隔
        animation_idle_left.set_interval(75);
        animation_idle_right.set_interval(75);
        animation_run_left.set_interval(75);
        animation_run_right.set_interval(75);
        animation_attack_ex_left.set_interval(75);
        animation_attack_ex_right.set_interval(75);
        animation_die_left.set_interval(75);
        animation_die_right.set_interval(75);

        // 取消循环播放
        animation_die_left.set_loop(false);
        animation_die_right.set_loop(false);
        // 角色尺寸
        size.x = 96, size.y = 96;

         // 大招状态定时器
        timer_attack_ex.set_wait_time(attack_ex_duration); // 持续时间
        timer_attack_ex.set_one_shot(true); // 单词触发
        timer_attack_ex.set_callback([&]() { // 大招回调成关闭
            is_attacking_ex = false;
        });

        timer_spwan_pea_ex.set_wait_time(100); // 大招子弹时间间隔
        timer_spwan_pea_ex.set_callback([&]() {// 大招期间生成速度更快的子弹
            spawn_pea_bullet(speed_pea_ex); // 传递速度
        });

        attack_cd = 100; // 平a cd
    }
    ~PeashooterPlayer() = default;
```

```
    void on_update(int delta) {

        Player::on_update(delta);

        if (is_attacking_ex) { // 大招屏幕抖动设置
            main_camera.shake(5, 100);
            timer_attack_ex.on_update(delta); // 大招攻击计时器
            timer_spwan_pea_ex.on_update(delta); // 豌豆大招持续时间;
        }
    }

    void on_attack() { // 平a
        spawn_pea_bullet(speed_pea);

        switch (rand() % 2)
        {
        case 0:
            mciSendString(_T("play pea_shoot_1 from 0"), NULL, 0, NULL);
            break;
        case 1:
            mciSendString(_T("play pea_shoot_2 from 0"), NULL, 0, NULL);
            break;
        }
    }

    void on_attack_ex() {
        is_attacking_ex = true;
        timer_attack_ex.restart(); // 重置大招计时器

        is_facing_right ? animation_attack_ex_right.reset() :
animation_attack_ex_left.reset(); // 根据角色朝向选择重置不同的角色动画状态

        mciSendString(_T("play pea_shoot_ex from 0"), NULL, 0, NULL);
    }

private:
    Timer timer_attack_ex; // 大招攻击计时器
    Timer timer_spwan_pea_ex; // 豌豆大招持续时间计时器

private:
    const float speed_pea = 0.75f; // 普攻速度
    const float speed_pea_ex = 1.5f; // 大招速度
    const int attack_ex_duration = 2500; // 大招持续时长
private:
    // 生产豌豆子弹
    void spawn_pea_bullet(float speed) { // 子弹对象速度大小
        Bullet* bullet = new PeaBullet();

        Vector2 bullet_position, bullet_velocity; // 子弹位置 速度
        const Vector2& bullet_size = bullet->get_size(); // 获得豌豆子弹的尺寸

        bullet_position.x = is_facing_right ? position.x + size.x - bullet_size.x /
2 : position.x - bullet_size.x / 2; // 计算发射位置，让他紧贴着豌豆的嘴
        bullet_position.y = position.y;
        bullet_velocity.x = is_facing_right ? speed : -speed; // 根据角色朝向设置速度
        bullet_velocity.y = 0; // 竖直速度分量设置为0

        // 设置到子弹对象
```

```
        bullet->set_position(bullet_position.x, bullet_position.y); // 位置
        bullet->set_velocity(bullet_velocity.x, bullet_velocity.y); // 速度

        bullet->set_collide_target(id == PlayerID::P1 ? PlayerID::P2 :
PlayerID::P1); // 子弹碰撞目标设置为对方角色

        bullet->set_callback([&]() { // 碰撞之后到回调函数，回调函数固定使怒气增加25
lambda中&表示捕获外部变量,按引用捕获
            mp += 25;
        });

        bullet_list.push_back(bullet); // 向全局数组添加子弹对象
    }

};
```

SunflowerPlayer 类 **：继承自 Player 类，实现龙日葵角色的功能，包括普攻、大招的释放和动画效果等操作。**

```
#include "player.h"
#include "sun_bullet.h"
#include "sun_bullet_ex.h"
#include <iostream>

extern Atlas atlas_sunflower_idle_left;
extern Atlas atlas_sunflower_idle_right;
extern Atlas atlas_sunflower_run_left;
extern Atlas atlas_sunflower_run_right;
extern Atlas atlas_sunflower_attack_ex_left;
extern Atlas atlas_sunflower_attack_ex_right;
extern Atlas atlas_sun_text;
extern Atlas atlas_sunflower_die_left;
extern Atlas atlas_sunflower_die_right;

extern Player* player_1;
extern Player* player_2;

class SunflowerPlayer : public Player{
public:
    SunflowerPlayer(bool facing_right = true) : Player(facing_right) { // 子类朝向传递
给父类
        // 获取动画集
        animation_idle_left.set_atlas(&atlas_sunflower_idle_left);
        animation_idle_right.set_atlas(&atlas_sunflower_idle_right);
        animation_run_left.set_atlas(&atlas_sunflower_run_left);
        animation_run_right.set_atlas(&atlas_sunflower_run_right);
        animation_attack_ex_left.set_atlas(&atlas_sunflower_attack_ex_left);
        animation_attack_ex_right.set_atlas(&atlas_sunflower_attack_ex_right);
        animation_sun_text.set_atlas(&atlas_sun_text);
        animation_die_left.set_atlas(&atlas_sunflower_die_left);
        animation_die_right.set_atlas(&atlas_sunflower_die_right);

        // 设置帧间隔
```

```
        animation_idle_left.set_interval(75);
        animation_idle_right.set_interval(75);
        animation_run_left.set_interval(75);
        animation_run_right.set_interval(75);
        animation_attack_ex_left.set_interval(100);
        animation_attack_ex_right.set_interval(100);
        animation_sun_text.set_interval(100);
        animation_die_left.set_interval(100);
        animation_die_right.set_interval(100);

        animation_attack_ex_left.set_loop(false); // 关闭技能循环播放
        animation_attack_ex_right.set_loop(false);
        animation_sun_text.set_loop(false);
        animation_die_left.set_loop(false);
        animation_die_right.set_loop(false);

        // 大招动画结束后
        animation_attack_ex_left.set_callback([&]() {
            is_attacking_ex = false;
            is_sun_text_visible = false;
        });

        animation_attack_ex_right.set_callback([&]() {
            is_attacking_ex = false;
            is_sun_text_visible = false;
            });

        // 角色尺寸
        size.x = 96, size.y = 96;

        attack_cd = 250; // 平a冷却
    }
    ~SunflowerPlayer() = default;

    void on_update(int delta) {

        Player::on_update(delta);

        if (is_sun_text_visible) animation_sun_text.on_update(delta); // 文本可见时更
新文本动画

    }

    void on_draw(const Camera& camera) {
        Player::on_draw(camera);

        if (is_sun_text_visible) { // 文本是否可见
            Vector2 text_position;
            IMAGE* frame = animation_sun_text.get_frame(); // 获取当前的动画帧
            // 计算取得向日葵头顶的“日”字坐标
            text_position.x = position.x - (size.x - frame->getwidth()) / 2;
            text_position.y = position.y - frame->getheight();
            animation_sun_text.on_draw(camera, (int)text_position.x,
(int)text_position.y);
        }
    }

    void on_attack()
```

```
    {
        Bullet* bullet = new SunBullet(); // 获得小型日光炸弹

        // 计算子弹位置(生成向日葵头部)
        Vector2 bullet_position;
        const Vector2& bullet_size = bullet->get_size();
        bullet_position.x = position.x + (size.x - bullet_size.x) / 2;
        bullet_position.y = position.y;

        // 设置位置和初速度
        bullet->set_position(bullet_position.x, bullet_position.y);
        bullet->set_velocity(is_facing_right ? velocity_sun.x : -velocity_sun.x,
velocity_sun.y);

        bullet->set_collide_target(id == PlayerID::P1 ? PlayerID::P2 :
PlayerID::P1); // 碰撞对象
        bullet->set_callback([&]() { // 子弹命中后奖励怒气值
            mp += 35;
        });

        bullet_list.push_back(bullet);
    }

    void on_attack_ex() { // 大招
        is_attacking_ex = true; // 打开大招状态
        is_sun_text_visible = true; // 打开文本状态

        animation_sun_text.reset(); // 重置文本计时器
        is_facing_right ? animation_attack_ex_right.reset() :
animation_attack_ex_left.reset(); // 根据朝向重置不同的角色本体动画

        Bullet* bullet = new SunBulletEx();
        Player* target_player = (id == PlayerID::P1 ? player_2 : player_1); // 根据
id判断目标玩家是哪一位

        Vector2 bullet_position, bullet_velocity; // 定义速度和位置变量来存储计算过程的中
间值

        const Vector2& bullet_size = bullet->get_size();
        const Vector2& target_size = target_player->get_size(); // 指向player类敌方玩
家位置
        const Vector2& target_position = target_player->get_position();
        // 计算生成位置
        bullet_position.x = target_position.x + (target_size.x - bullet_size.x) / 2;
        bullet_position.y = -size.y;
        // 计算下落速度
        bullet_velocity.x = 0;
        bullet_velocity.y = speed_sun_ex;
        // 设置大招位置 和 速度
        bullet->set_position(bullet_position.x, bullet_position.y);
        bullet->set_velocity(bullet_velocity.x, bullet_velocity.y);

        bullet->set_collide_target(id == PlayerID::P1 ? PlayerID::P2 :
PlayerID::P1); // 碰撞目标

        bullet->set_callback([&]() { // 给予命中后怒气奖励
            mp += 50;
        });
```

```
        bullet_list.push_back(bullet);

        mciSendString(_T("play sun_text from 0"), NULL, 0, NULL);
    }


private:
    const float speed_sun_ex = 0.15f; // 大型日光炸弹下落速度
    const Vector2 velocity_sun = { 0.25f, -0.5f }; // 小型日光炸弹抛射速度
private:
    Animation animation_sun_text; // 头顶文本动画
    bool is_sun_text_visible = false; // 是否显示头顶文本

};
```

3.2.4 子弹类

`Bullet 类`：抽象基类，定义了子弹的基本接口，包括子弹的更新、绘制、碰撞检测和销毁等操作。

```
#include "vector2.h"
#include "player_id.h"
#include "camera.h"
#include "bullet.h"

#include <functional>
#include <graphics.h>

extern bool is_debug;

class Bullet {
public:
    Bullet() = default;
    ~Bullet() = default;

    void set_damge(int val) { // 设置子弹对象所造成的伤害
        damage = val;
    }

    int get_damge() { // 获取子弹对象可以造成的伤害
        return damage;
    }

    void set_position(float x, float y) { // 设置子弹位置
        position.x = x, position.y = y;
    }

    const Vector2& get_positsion() const { // 获取子弹位置
        return position;
    }

    const Vector2& get_size() const { // 获取子弹尺寸
        return size;
    }
```

```
    void set_velocity(float x, float y) { // 设置子弹速度
        velocity.x = x, velocity.y = y;
    }

    void set_collide_target(PlayerID target) { // 设置碰撞目标
        target_id = target;
    }

    PlayerID get_collide_target() const{ // 获取碰撞目标id
        return target_id;
    }

    void set_callback(std::function<void()> callback) { // 子弹碰撞后调用回调函数
        this->callback = callback;
    }

    void set_valid(bool flag) { // 设置子弹是否继续碰撞
        valid = flag;
    }

    bool get_valid() const { // 获取子弹是否继续碰撞
        return valid;
    }

    bool check_can_remove() const{ // 获取子弹移除
        return can_remove;
    }

    virtual void on_collide() { // 子弹碰撞时调用
        if (callback) callback();

    }

    // 检测子弹是否与指定位置发生了碰撞
    virtual bool check_collision(const Vector2& position, const Vector2& size) { //
待测举矩形的左上角顶点 子弹的尺寸
        return this->position.x + this->size.x / 2 ≥ position.x // 计算子弹(this指向)
中心 看看是不是比矩形的左上角顶点大
            && this->position.x + this->size.x / 2 ≤ position.x + size.x
            && this->position.y + size.y / 2 ≥ position.y
            && this->position.y + size.y / 2 ≤ position.y + size.y;
    }

    virtual void on_update(int delta) { } // 内部执行子弹更新
    virtual void on_draw(const Camera& camera) const{ // 内部执行子弹绘制

        if (is_debug) {
            // 由于小型子弹都是中心点判定，大型是边框判断，所以额外增加调试信息
            setfillcolor(RGB(255, 255, 255));
            setlinecolor(RGB(255, 255, 255));
            // 矩形边缘线和中心线坐标实心圆构成
            rectangle((int)position.x, (int)position.y, (int)(position.x + size.x),
(int)(position.y + size.y));
            solidcircle((int)(position.x + size.x / 2), (int)(position.y + size.y /
2), 5);
        }
    }
```

```
protected:
    Vector2 size; // 子弹尺寸
    Vector2 position; // 子弹位置
    Vector2 velocity; // 子弹速度
    int damage = 10; // 子弹伤害

    bool valid = true; // 子弹是否有效
    bool can_remove = false; // 子弹是否可以被移除
    std::function<void()> callback; // 子弹碰撞回调

    PlayerID target_id = PlayerID::P1; // 子弹碰撞玩家id

protected:

    bool check_if_exceeds_screen() { // 检查是否超出屏幕
        // 0 ≤ 左顶点位置 + 长度 ≤边界
        return (position.x + size.x ≤ 0 || position.x + size.x ≥ getwidth() ||
position.y + size.y ≤ 0 || position.y ≥ getheight());
    }


};
```

PeaBullet 类 **：继承自 Bullet 类，实现豌豆子弹的功能，包括子弹的移动、碰撞检测和破碎动画等操作。**




```
#include "bullet.h"
#include "animation.h"
#include "camera.h"
#include "atlas.h"

extern IMAGE img_pea;
extern Atlas atlas_pea_break;

class PeaBullet : public  Bullet{
public:
    PeaBullet() {
        size.x = 64, size.y = 64;
```

```
        damage = 10;

        animation_break.set_atlas(&atlas_pea_break); // 设置图集
        animation_break.set_interval(100); // 帧间隔
        animation_break.set_loop(false); // 是否循环
        animation_break.set_callback([&]() { // 动画结束后将子弹设置为可被移除状态
            can_remove = true;
        });

    }
    ~PeaBullet() = default;

    void on_collide() {
        Bullet::on_collide();

        switch (rand() % 3)
        {
        case 0:
            mciSendString(_T("play pea_break_1 from 0"), NULL, 0, NULL);
            break;
        case 1:
            mciSendString(_T("play pea_break_2 from 0"), NULL, 0, NULL);
            break;
        case2:
            mciSendString(_T("play pea_break_3 from 0"), NULL, 0, NULL);
            break;
        }

    }

    void on_update(int delta) { // 修改子弹位置
        position += velocity * (float)delta;
        if (!valid) animation_break.on_update(delta); // 如果失效就更新破碎动画
        if (check_can_remove()) can_remove = true; // 如果飞出屏幕就移除
    }

    void on_draw(const Camera& camera) const{
        if (valid) putimage_alpha(camera, (int)position.x, (int)position.y,
&img_pea); // 如果有效就绘制未破碎动画
        else animation_break.on_draw(camera, (int)position.x, (int)position.y); //
失效就播放破碎动画

        Bullet::on_draw(camera);
    }



private:
    Animation animation_break; // 豌豆子弹破碎动画
};
```

SunBullet 类 **：继承自 Bullet 类，实现小型日光炸弹的功能，包括子弹的移动、碰撞检测和爆炸动画等操作。**





```cpp
#include "bullet.h"
#include "animation.h"
#include "atlas.h"
#include "camera.h"

extern Atlas atlas_sun;
extern Atlas atlas_sun_explode;

extern Camera main_camera;

class SunBullet : public Bullet {
public:
    SunBullet() {
        size.x = 96, size.y = 96;

        damage = 10;

        // 小日光动画
        animation_idle.set_atlas(&atlas_sun); // 播放动画
        animation_idle.set_interval(50);
        // 爆炸动画
        animation_explode.set_atlas(&atlas_sun_explode); // 播放动画
        animation_explode.set_interval(50);
        animation_explode.set_loop(false); // 不循环
        animation_explode.set_callback([&]() { // 清除
```

```
            can_remove = true;
        });

        // 获取图集中每一帧的图片
        IMAGE* frame_idle = animation_idle.get_frame();
        IMAGE* frame_explode = animation_explode.get_frame();
        // 计算中心对齐做对应的偏移量
        explode_render_offset.x = (frame_idle->getwidth() - frame_explode-
>getwidth()) / 2.0f;
        explode_render_offset.y = (frame_idle->getheight() - frame_explode-
>getheight()) / 2.0f;
    }
    ~SunBullet() = default;

    void on_collide() {
        Bullet::on_collide(); // 重写on_collide()函数

        main_camera.shake(5, 250); // 抖动强度 时间
        mciSendString(_T("play sun_explode from 0"), NULL, 0, NULL);

    }

    void on_update(int delta) {
        if (valid) { // 有效
            // 计算重力
            velocity.y += gravity * delta; // v = v0 + gt
            position += velocity * (float)delta; // x = x0 + v*t
        }
        if (!valid) animation_explode.on_update(delta); // 失效了
        else animation_idle.on_update(delta); // 有效

        if (check_can_remove()) can_remove = true; // 飞出屏幕标记为可移除状态

    }

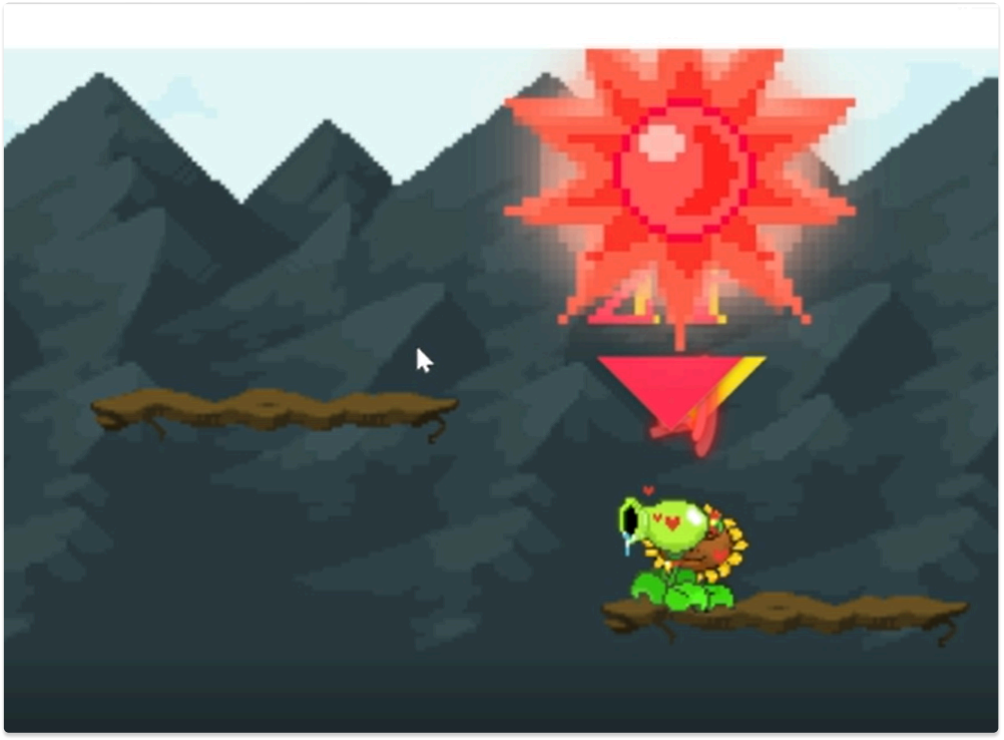

    void on_draw(const Camera& camera) const{
        if (valid) animation_idle.on_draw(camera, (int)position.x, (int)position.y);
// 有效 绘制日光
        else animation_explode.on_draw(camera, (int)(position.x +
explode_render_offset.x), (int)(position.y + explode_render_offset.y)); // 无效 绘制
爆炸

        Bullet::on_draw(camera);
    }

private:
    const float gravity = 1e-3f; // 日光炸弹重力

private:
    Animation animation_idle; // 日光默认动画
    Animation animation_explode; // 日光爆炸动画
    Vector2 explode_render_offset; // 爆炸动画渲染偏移(easyx中渲染一直是矩形左上角坐标，但
是要与爆炸中心对其，所以要有一个偏移)
};
```

SunBulletEx 类 ：继承自 Bullet 类，实现大型日光炸弹的功能，包括子弹的移动、碰撞检测和爆炸动画等操作。






```
#include "bullet.h"
#include "animation.h"
```

```cpp
extern Atlas atlas_sun_ex;
extern Atlas atlas_sun_ex_explode;

extern Camera main_camera;

class SunBulletEx : public Bullet{
public:
    SunBulletEx() {
        size.x = 270, size.y = 270; // 判定范围

        damage = 30;

        // 大日光动画
        animation_idle.set_atlas(&atlas_sun_ex); // 播放动画
        animation_idle.set_interval(50);
        // 爆炸动画
        animation_explode.set_atlas(&atlas_sun_ex_explode); // 播放动画
        animation_explode.set_interval(50);
        animation_explode.set_loop(false); // 不循环
        animation_explode.set_callback([&]() { // 清除
            can_remove = true;
        });

        // 获取图集中每一帧的图片
        IMAGE* frame_idle = animation_idle.get_frame();
        IMAGE* frame_explode = animation_explode.get_frame();
        // 计算中心对齐做对应的偏移量
        explode_render_offset.x = (frame_idle->getwidth() - frame_explode-
>getwidth()) / 2.0f;
        explode_render_offset.y = (frame_idle->getheight() - frame_explode-
>getheight()) / 2.0f;
    }

    ~SunBulletEx() = default;


    void on_collide() {
        Bullet::on_collide(); // 重写on_collide()函数

        main_camera.shake(20, 350); // 抖动强度 时间
        mciSendString(_T("play sun_explode_ex from 0"), NULL, 0, NULL);
    }

    // 大型日光设置为接触边缘有伤害
    bool check_collision(const Vector2& position, const Vector2& size) // 目标位置 技
能尺寸
    {
        // 两个矩形是否相交
        bool is_collide_x = (max(this->position.x + this->size.x, position.x +
size.x) - min(this->position.x, position.x) <= this->size.x + size.x);
        bool is_collide_y = (max(this->position.y + this->size.y, position.y +
size.y) - min(this->position.y, position.y) <= this->size.y + size.y);

        return is_collide_x && is_collide_y;
    }

    void on_update(int delta) {
        if (valid) position += velocity * (float)delta; //  子弹有效时更新大日光的位置
```

```
        if (!valid) animation_explode.on_update(delta); // 如果失效就更新破碎动画
        else animation_idle.on_update(delta); // 如果有效更新默认动画

        if (check_can_remove()) can_remove = true; // 如果飞出屏幕就移除

    }

    void on_draw(const Camera& camera) const {
        if (valid) animation_idle.on_draw(camera, (int)position.x, (int)position.y);
// 有效 绘制日光
        else animation_explode.on_draw(camera, (int)(position.x +
explode_render_offset.x), (int)(position.y + explode_render_offset.y)); // 无效 绘制
爆炸

        Bullet::on_draw(camera);
    }

private:
    Animation animation_idle; // 大日光默认动画
    Animation animation_explode; // 大日光爆炸动画
    Vector2 explode_render_offset; // 爆炸渲染偏移

};
```

3.2.5 其他类

Atlas 类 ：图集类，用于加载和管理游戏中的图片资源，实现动画的播放和切换。

```
// 图集类
#include <vector>
#include <graphics.h>

class Atlas {
public:
    Atlas() = default;
    ~Atlas() = default;

    void load_from_file(LPCTSTR path_template, int num) { // 加载图片到类中
        img_list.clear();
        img_list.resize(num);

        TCHAR path_file[256];
        for (int i = 0; i < num; i++) {
            _stprintf_s(path_file, path_template, i + 1);
            loadimage(&img_list[i], path_file);
        }
    }

    void clear() { // 清空图集中已加载的图片对象
        img_list.clear();
    }

    int get_size() { // 获取图集中图片的数量
        return (int)img_list.size();
```

```
    }

    IMAGE* get_image(int idx) { // 获取实际渲染的动画帧
        if (idx < 0 || idx ≥ img_list.size()) return nullptr; // 越界返回空指针
        return &img_list[idx]; // 返回地址
    }

    void add_image(const IMAGE& img) { // 向图集中添加图片对象
        img_list.push_back(img);
    }

private:
    std::vector<IMAGE> img_list;



};
```

Animation 类 **：动画类，用于控制动画的播放、暂停和重置等操作，实现角色和技能的动画效果。**

```
#include <iostream>
#include <functional>
#include <graphics.h>
#include "atlas.h"
#include "unil.h"
#include "camera.h"

// 动画类
class Animation {
public:
    Animation() = default;
    ~Animation() = default;

    void reset() { // 重置动画状态
        timer = 0;
        idx_frame = 0;
    }

    void set_atlas(Atlas* new_atlas) { // 设置动画使用图集
        reset(); // 重置状态防止越界
        atlas = new_atlas;
    }

    void set_loop(bool flag) { // 设置动画是否循环播放
        is_loop = flag;
    }

    void set_interval(int ms) { // 设置动画帧间隔
        interval = ms;
    }

    int get_idx_frame() { // 获取当前帧索引
        return idx_frame;
    }
```

```
    IMAGE* get_frame() { // 获取当前帧图片
        return atlas->get_image(idx_frame);
    }

    bool check_finished() { // 检查动画是否播放结束
        if (is_loop) return false; //  如果循环就永不结束
        return (idx_frame == atlas->get_size() - 1); // 检查当前帧索引是否到尾部
    }

    void set_callback(std::function<void()> callback) { // 回调函数
        this->callback = callback;
    }

    void on_update(int delta) { // 这个函数全跑完为一帧

        timer += delta; // 每次更新动画增加计时器的值
        if (timer >= interval) { // 检查计时器是否到达帧间隔
            timer = 0;
            idx_frame++; // 到达的话累加帧索引
            if (idx_frame >= atlas->get_size()) {
                // 根据动画是否循环播放设置动画帧是从头开始还是保持在图集最后位置上
                idx_frame = is_loop ? 0 : atlas->get_size() - 1; // 检查动画是否到图集
尾部
                if (!is_loop && callback) callback(); // 到了尾部，并且动画没有循环播放，
且回调函数存在则执行回调函数
            }
        }
    }

    void on_draw(const Camera& camera, int x, int y) const {
        putimage_alpha(camera, x, y, atlas->get_image(idx_frame));
    }




private:
    int timer = 0; // 计时器
    int interval = 0; // 帧间隔
    int idx_frame = 0; // 帧索引
    bool is_loop = true; // 是否循环
    Atlas* atlas = nullptr;
    std::function<void()> callback;
};
```

Platform 类 ：平台类，用于管理游戏中的平台，包括平台的绘制和碰撞检测等操作。

```
#include "unil.h"
#include "camera.h"

extern bool is_debug;

class Platform {
public:
    struct  CollisionShape {
```

```
        float y; // 平台的竖直坐标
        float left, right; // 水平方向平台的左右边缘

    };
public:
    CollisionShape shape;
    IMAGE* img = nullptr;
    POINT render_positon = { 0 }; // Colisionshape 不是已经记录了坐标信息，为什么还要单独记录渲染位置呢？ 因为平台素材是有厚度的顶部不一定是碰撞线所处的位置
public:
    Platform() = default;
    ~Platform() = default;

    void on_draw(const Camera& camera) const {
        // 绘制自身
        putimage_alpha(camera, render_positon.x, render_positon.y, img);
        if (is_debug) {
            // 绘制水平线
            setlinecolor(RGB(255, 0, 0));
            line(camera, (int)shape.left, (int)shape.y, (int)shape.right, (int)shape.y);
        }
    }

private:

};
```

StatusBar 类 ：状态栏类，用于显示角色的生命值和能量值，实现游戏状态的可视化。

```
#include "unil.h"

class StatusBar {
public:
    StatusBar() = default;
    ~StatusBar() = default;

    void set_avater(IMAGE* img) { // 设置显示图片
        img_avater = img;
    }

    void set_position(int x, int y) { // 设置状态栏位置
        position.x = x, position.y = y;
    }
     // 设置生命和蓝条
    void set_hp(int val) {
        hp = val;
    }

    void set_mp(int val) {
        mp = val;
    }

    void on_draw() { // 因为状态栏不需要摄像机位置来渲染,所以不用添加参数
        putimage_alpha((int)position.x, (int)position.y, img_avater); // 渲染头像图片
```

```
        // 使用不同的绘图颜色，绘制圆角矩形两次
        setfillcolor(RGB(5, 5, 5));
        solidroundrect(position.x + 100, position.y + 10, position.x + 100 + width +
3 * 2, position.y + 36, 8, 8);
        solidroundrect(position.x + 100, position.y + 45, position.x + 100 + width +
3 * 2, position.y + 71, 8, 8);
        setfillcolor(RGB(67, 47, 47));
        solidroundrect(position.x + 100, position.y + 10, position.x + 100 + width +
3, position.y + 33, 8, 8);
        solidroundrect(position.x + 100, position.y + 45, position.x + 100 + width +
3, position.y + 68, 8, 8);

        // 状态条填充宽度的计算
        float hp_bar_width = width * max(0, hp) / 200.0f; // 保证血量不为负数
        float mp_bar_width = width * min(200, mp) / 200.0f;
         // 填充状态栏
        setfillcolor(RGB(197, 61, 67));
        solidroundrect(position.x + 100, position.y + 10, position.x + 100 +
(int)hp_bar_width + 3, position.y + 33, 8, 8);
        setfillcolor(RGB(83, 131, 195));
        solidroundrect(position.x + 100, position.y + 45, position.x + 100 +
(int)mp_bar_width + 3, position.y + 68, 8, 8);
    }

private:
    const int width = 275; // 状态栏宽度
    int hp = 0; // 显示的生命值
    int mp = 0; //  显示的蓝条

    POINT position = { 0 }; // 窗口中状态栏显示的位置
    IMAGE* img_avater = nullptr; // 角色头像
};
```

3.3 模块交互设计

游戏各模块之间通过消息传递和函数调用的方式进行交互，具体如下：

场景管理模块 ：负责调用其他模块的相应函数，实现场景的切换和管理。例如，在游戏场景中，调用角色管理模块和子弹管理模块的更新和绘制函数，实现角色和子弹的动态显示。

角色管理模块 ：接收玩家的输入信息，更新角色的状态和位置，并调用动画管理模块的函数，实现角色的动画效果。同时，角色管理模块还会与子弹管理模块和平台管理模块进行交互，处理角色与子弹、角色与平台之间的碰撞检测。

子弹管理模块 ：负责子弹的发射、移动和碰撞检测等操作。当子弹与角色发生碰撞时，子弹管理模块会调用角色管理模块的函数，更新角色的生命值和能量值。

平台管理模块 ：负责平台的绘制和碰撞检测等操作。当角色与平台发生碰撞时，平台管理模块会调用角色管理模块的函数，更新角色的位置和速度。

动画管理模块 ：为角色和技能提供动画效果，根据角色和技能的状态，调用 Atlas 类的函

数，实现动画的播放和切换。

音效管理模块 ：根据游戏的不同状态，播放相应的音效，如背景音乐、攻击音效、技能音效等，提升游戏的沉浸感。

四、详细设计与实现

4.1 资源加载

游戏在启动时，会调用 load_game_resources 函数加载所需的图片和音频资源，包括角色图片、背景图片、子弹图片、音效文件等。

```
void load_game_resources() {

// 加载游戏字体

AddFontResourceEx(_T("resources/IPix.ttf"), FR_PRIVATE, NULL);

// 加载图片资源

loadimage(&img_menu_background, _T("resources/menu_background.png"));

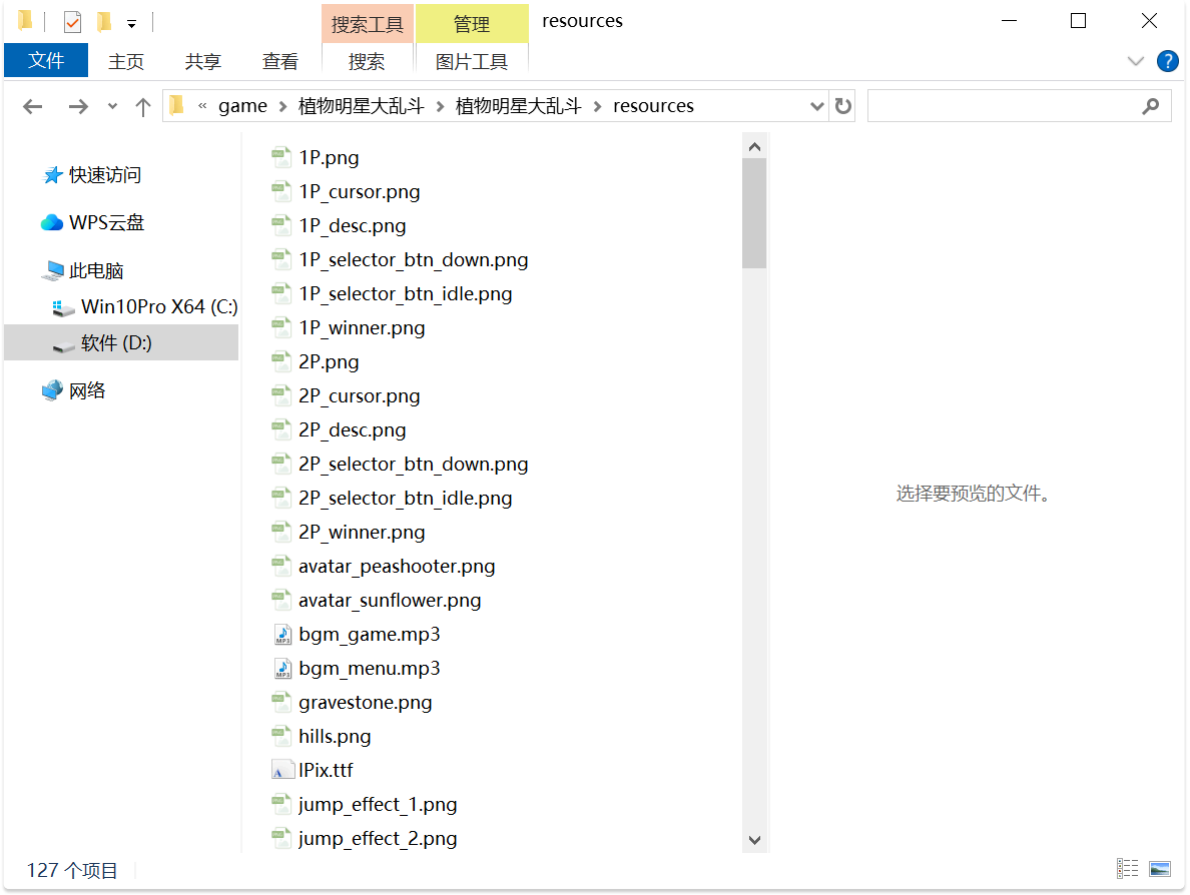
// ...

// 加载音频资源

mciSendString(_T("open resources/bgm_game.mp3 alias bgm_game"), NULL, 0, NULL);

mciSendString(_T("open resources/bgm_menu.mp3 alias bgm_menu"), NULL, 0, NULL);

// ...}
```



4.2 场景管理

游戏通过 SceneManager 类实现场景的管理和切换。SceneManager 类包含一个当前场景指针，通过调用场景的 on_enter、on_update、on_draw、on_input 和 on_exit 函数，实现场景的初始化、更新、绘制、输入处理和卸载等操作。

```
// 场景管理类
#include "scene.h"

extern Scene* menu_scene;
extern Scene* game_scene;
extern Scene* selector_scene;

class SceneManager {
public:

    enum class SceneType {
        Menu,
        Game,
        Selector
    };

public:

    SceneManager() = default;
    ~SceneManager() = default;

    void set_current_scene(Scene* scene) {
```

```
        current_scene = scene;
        current_scene->on_enter();
    }

    void switch_to(SceneType type) {
        current_scene->on_exit();
        switch (type)
        {
        case SceneType::Menu:
            current_scene = menu_scene;
            break;
        case SceneType::Game:
            current_scene = game_scene;
            break;
        case SceneType::Selector:
            current_scene = selector_scene;
            break;
        default:
            break;
        }
        current_scene->on_enter();
    }

    void on_update(int delta) { // 数据处理
        current_scene->on_update(delta);
    }

    void on_draw(const Camera& camera) { // 渲染场景
        current_scene->on_draw(camera);
    }

    void on_input(const ExMessage& msg) { // 处理场景输入
        current_scene->on_input(msg);
    }

private:
    Scene* current_scene = nullptr;
};
```

4.3 角色控制

玩家可以通过键盘控制角色的移动、跳跃、攻击和释放技能等动作。在 Player 类的 on_input 函数中，根据玩家的输入信息，更新角色的状态和位置。

```
virtual void on_input(const ExMessage& msg) {
    switch (msg.message)
    {
    case WM_KEYDOWN: // 按下把方向左右执行
        switch (id)
        {
        case PlayerID::P1:
            switch (msg.vkcode)
            {
                //'A'
            case 0x41:
                is_left_key_down = true;
```

```
            break;
            // 'D'
        case 0x44:
            is_right_key_down = true;
            break;
            // 'W'
        case 0x57:
            //is_w_key_down = true;
            on_jump();
            break;
            //'F'
        case 0x46:
            if (can_attack) {
                on_attack();
                can_attack = false; // 设置为不可以放普通攻击
                timer_attack_cd.restart(); // 重置cd
            }
            break;
        // 'G'
        case 0x47:
            if (mp ≥ 100) { // 如果满怒气条了
                on_attack_ex();
                mp = 0;
            }
        }
        break;
    case PlayerID::P2:
        switch (msg.vkcode)
        {
            //'←'
        case VK_LEFT:
            is_left_key_down = true;
            break;
            // '→'
        case VK_RIGHT:
            is_right_key_down = true;
            break;

        case VK_UP:
            //is_up_key_down = true;
            on_jump();
            break;
            // '.'
        case VK_OEM_PERIOD:
            if (can_attack) {
                on_attack();
                can_attack = false; // 设置为不可以放普通攻击
                timer_attack_cd.restart(); // 重置cd
            }
            break;
            //'/'
        case VK_OEM_2:
            if (mp ≥ 100) {
                on_attack_ex();
                mp = 0;
            }
        }
    }
```

```
        break;
    case WM_KEYUP: // 抬起关闭左右变量
        switch (id)
        {
        case PlayerID::P1:
            switch (msg.vkcode)
            {
                //'A'
            case 0x41:
                is_left_key_down = false;
                break;
                // 'D'
            case 0x44:
                is_right_key_down = false;
                break;
            //case 0x57:
            //  is_w_key_down = false;
            //  break;
            }
            break;

        case PlayerID::P2:
            switch (msg.vkcode)
            {
                //'A'
            case VK_LEFT:
                is_left_key_down = false;
                break;
                // 'D'
            case VK_RIGHT:
                is_right_key_down = false;
                break;
            //case VK_UP:
            //  is_up_key_down = false;
            //  break;
            }
            break;
        }
        break;

    }
}
```

在 Player 类的 on_update 函数中，根据角色的移动状态和速度，更新角色的位置。

```
void Player::on_update(int delta) {

int direction = is_right_key_down - is_left_key_down;

if (direction != 0) {

if (!is_attacking_ex) is_facing_right = direction > 0;

float distance = direction * run_velocity * delta;
```

```
on_run(distance);

} else {

timer_run_effect_generation.pause();

}

// ...}
```

4.4 碰撞检测

碰撞检测是游戏中的重要部分，用于判断角色与子弹、角色与平台之间的碰撞关系。

4.4.1 角色与子弹的碰撞检测

在 Bullet 类的 check_collision 函数中，判断子弹是否与角色发生碰撞。

```
virtual bool check_collision(const Vector2& position, const Vector2& size) { // 待测举矩形的左上角顶点 子弹的尺寸
    return this->position.x + this->size.x / 2 ≥ position.x // 计算子弹(this指向)中心看看是不是比矩形的左上角顶点大
        && this->position.x + this->size.x / 2 ≤ position.x + size.x
        && this->position.y + size.y / 2 ≥ position.y
        && this->position.y + size.y / 2 ≤ position.y + size.y;
}
```

在 Player 类的 move_and_collide 函数中，遍历子弹列表，检测角色与子弹的碰撞。

```
            if(!is_invulnerable) { // 如果不处于无敌状态
                for (Bullet* bullet : bullet_list) {
                    if (!bullet->get_valid() || bullet->get_collide_target() ≠ id) continue; // 当前子弹不可以发生碰撞 碰撞目标不能是发射者自己

                    if (bullet->check_collision(position, size)) { // 如果可以发生碰撞
                        make_invulnerable(); // 如果进入就进入无敌状态
                        bullet->on_collide(); // 碰撞就调用回调函数
                        bullet->set_valid(false); // 设置这个子弹不可以继续发生碰撞
                        hp -= bullet->get_damge(); // 减去这个子弹所造成的伤害

                        // 子弹位置与玩家位置进行做差，更新玩家最后一次受击方向
                        last_hurt_direction = bullet->get_positsion() - position;
                        if (hp ≤ 0) { // 没血了设置尸体起飞水平速度和数值速度
                            velocity.x = last_hurt_direction.x < 0 ? 0.35f : -0.35f;
                            velocity.y = -1.0f;
                        }
                    }
```

```
                }
            }

        }
```

4.4.2 角色与平台的碰撞检测

在 Player 类的 move_and_collide 函数中，遍历平台列表，检测角色与平台的碰撞。

```
            float last_velocity_y = velocity.y; // 记录上一帧的速度
             // 设置重力
            velocity.y += gravity * delta; // 根据重力加速度的值对玩家的速度进行累加
            position += velocity * (float)delta; // 根据速度值对玩家的位置更新

            if (hp ≤ 0) return; // 防止角色在死亡过程中与场景中的障碍物发生判断

            if (velocity.y > 0) {
                for (const auto& platform : platform_list) {
                    const Platform::CollisionShape& shape = platform.shape; // 平台的
碰撞外形
                    // 图形是否在水平方向上重合只需要去二者最右边的值和二者最左边的值进行做
差，如果结果小于两个宽度之和，说明这两个图形在水平方向上有重合部分
                    bool is_collde_x = (max(position.x + size.x, shape.right) -
min(position.x, shape.left) ≤ size.x + (shape.right - shape.left));
                    bool is_collde_y = (shape.y ≥ position.y && shape.y ≤
position.y + size.y);

                    if (is_collde_x && is_collde_y) { // 水平方向和竖直方向都发生碰撞
                        float delta_pos_y = velocity.y * delta; // 这一帧角色在数值方向
上移动的距离
                        float last_tick_foot_pos_y = position.y + size.y -
delta_pos_y; // 计算上一帧玩家脚底的位置 角色位置 + 角色尺寸 - 这一帧的距离
                        if (last_tick_foot_pos_y ≤ shape.y) { // 前一帧玩家是否整个穿
过了平台
                            position.y = shape.y - size.y;
                            velocity.y = 0; // 重置y方向速度

                            if (last_velocity_y ≠ 0) on_land(); // 与平台碰撞并且上一
帧速度不是0的时候，触发落地特效

                            break;
                        }
                    }
                }
            }
```

4.5 动画效果

动画效果是游戏的重要组成部分，用于增强游戏的视觉表现力。



4.5.1 角色动画

角色动画包括闲置、行走、攻击、死亡等状态。在 Player 类中，根据角色的状态，设置当前动画。



```
    virtual void on_update(int delta) {
        int direction = is_right_key_down - is_left_key_down; // 通过做差的方式判断移动
方向
```

```
        if (direction ≠ 0) {
            if(!is_attacking_ex) is_facing_right = direction > 0; // 角色开大不能转向
            current_animation = is_facing_right ? &animation_run_right :
&animation_run_left;
            float distance = direction * run_velocity * delta; // 向量 * 速度 * 时间
            on_run(distance);
        }else {
            current_animation = is_facing_right ? &animation_idle_right :
&animation_idle_left;
            timer_run_effect_generation.pause(); // 停止跑动特效生成
        }
        // 开大变化成大招动画
        if (is_attacking_ex) current_animation = is_facing_right ?
&animation_attack_ex_right : &animation_attack_ex_left;
        // 死亡的时候
        if (hp ≤ 0) current_animation = last_hurt_direction.x < 0 ?
&animation_die_left : &animation_die_right; // 根据受击方向觉得角色飞行方向

        current_animation->on_update(delta);
        animation_jump_effect.on_update(delta);
        animation_land_effect.on_update(delta);

        timer_attack_cd.on_update(delta); // 更新计时器
        timer_invulnerable.on_update(delta);
        timer_invulnerable_blink.on_update(delta);
        timer_run_effect_generation.on_update(delta); // 添加定时器对象
        timer_cursor_visibility.on_update(delta);

        if (hp ≤ 0) timer_die_effect_generation.on_update(delta); // 没血了就添加死亡
计时器对象

        particle_list.erase(std::remove_if( // 清除用过的特效
            particle_list.begin(), particle_list.end(),
            [](const Particle& particle)
            {
                return !particle.check_valid(); // 如果粒子无效了就取反变成true执行删除逻
辑
            }),
            particle_list.end());
        for (Particle& particle : particle_list) particle.on_update(delta); // 遍历执
行特效

        if (is_showing_sketch_frame) sketch_image(current_animation->get_frame(),
&img_sketch); // 获取当前帧图片
        move_and_collide(delta); // 物理场景设置 + 攻击
    }
```

在 Player 类的 on_draw 函数中，绘制当前动画。

```
virtual void on_draw(const Camera& camera) {
    if (is_jump_effect_visible) animation_jump_effect.on_draw(camera,
(int)position_jump_effect.x, (int)position_jump_effect.y);
    if (is_land_effect_visible) animation_land_effect.on_draw(camera,
(int)position_land_effect.x, (int)position_land_effect.y);
```

```
    for (const Particle& particle : particle_list) particle.on_draw(camera); // 遍历
绘制特效

    if (hp > 0 && is_invulnerable && is_showing_sketch_frame) putimage_alpha(camera,
(int)position.x, (int)position.y, &img_sketch);
    else current_animation->on_draw(camera, (int)position.x, (int)position.y);

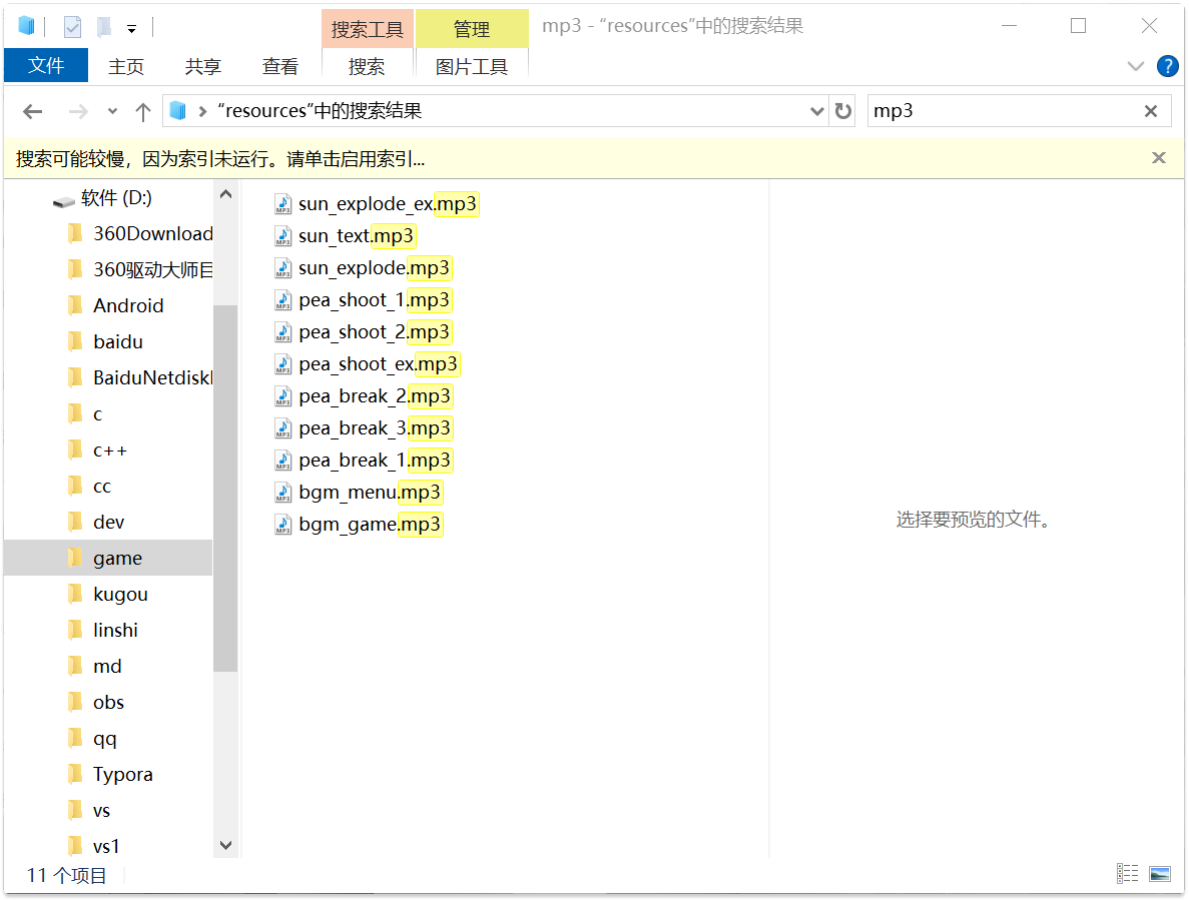
    if (is_cursor_visible) { //  光标可见
        switch (id)
        {
        case PlayerID::P1: // 角色光标跟随摄像机渲染在角色头顶
            putimage_alpha(camera, (int)(position.x + (size.x -
img_1P_cursor.getwidth()) / 2), (int)(position.y - img_1P_cursor.getheight()),
&img_1P_cursor);
            break;
        case PlayerID::P2:
            putimage_alpha(camera, (int)(position.x + (size.x -
img_2P_cursor.getwidth()) / 2), (int)(position.y - img_2P_cursor.getheight()),
&img_2P_cursor);
            break;
        }
    }

    if (is_debug) { //  开启 debug 绘制轮廓线
        setlinecolor(RGB(0, 125, 255));
        rectangle((int)position.x, (int)position.y, (int)(position.x + size.x),
(int)(position.y + size.y)); // 在窗口坐标下绘制线框
    }

}
```

4.6 音效处理

音效处理是游戏的重要组成部分，用于提升游戏的沉浸感。



4.6.1 背景音乐

在 MenuScene 类的 on_enter 函数中，播放背景音乐。

```
    void on_enter() {
        mciSendString(_T("play bgm_menu repeat from 0"), NULL, 0, NULL);
    }
```

在 GameScene 类的 on_enter 函数中，播放背景音乐。

```
void GameScene::on_enter() {

mciSendString(_T("play bgm_game repeat from 0"), NULL, 0, NULL);}
```

4.6.2 攻击音效

在 Player 类的 on_attack 函数中，播放攻击音效。

```
void Player::on_attack() {

switch (rand() % 2) {

case 0:

mciSendString(_T("play pea_shoot_1 from 0"), NULL, 0, NULL);

break;
```

```
case 1:

mciSendString(_T("play pea_shoot_2 from 0"), NULL, 0, NULL);

break;

}}
```

4.6.3 技能音效

在 Player 类的 on_attack_ex 函数中，播放技能音效。

```
void Player::on_attack_ex() {

mciSendString(_T("play sun_text from 0"), NULL, 0, NULL);}
```

4.6.4 碰撞音效

在 Bullet 类的 on_collide 函数中，播放碰撞音效。

```
void Bullet::on_collide() {

mciSendString(_T("play sun_explode from 0"), NULL, 0, NULL);}
```

4.7 游戏结算

游戏结算是在一方角色的生命值归零时，显示获胜玩家的信息。



4.7.1 结算逻辑

在 GameScene 类的 on_update 函数中，判断游戏是否结束。

```
void on_update(int delta) {
    player_1->on_update(delta);
    player_2->on_update(delta);

    main_camera.on_update(delta); // 更新主摄像机对象

    // 子弹移除逻辑
    bullet_list.erase(std::remove_if(
        bullet_list.begin(), bullet_list.end(),
        [](const Bullet* bullet) {
            bool detetable = bullet->check_can_remove(); // 如果子弹可以移除就返回true
            if (detetable) delete bullet; // 释放子弹内存
            return detetable;
        }),
        bullet_list.end());

    for (Bullet* bullet : bullet_list) bullet->on_update(delta); // 遍历每一个子弹对象(更新子弹)

    // 获得玩家的位置
    const Vector2& position_player_1 = player_1->get_position();
    const Vector2& position_player_2 = player_2->get_position();
    // 如果玩家掉出世界
    if (position_player_1.y >= getheight()) player_1->set_hp(0); //  坐标超出窗口下方，生命值为0
    if (position_player_2.y >= getheight()) player_2->set_hp(0);

    if (player_1->get_hp() <= 0 || player_2->get_hp() <= 0) { // 生命归0
        if (!is_game_over) { // 游戏结束的最后一帧结束背景音乐，播放win音乐
            mciSendString(_T("stop bgm_game"), NULL, 0, NULL);
            mciSendString(_T("play ui_win from 0"), NULL, 0, NULL);
        }
        is_game_over = true; // 游戏结束
    }
```

```
        // 获得玩家的状态
        status_bar_1P.set_hp(player_1->get_hp());
        status_bar_1P.set_mp(player_1->get_mp());
        status_bar_2P.set_hp(player_2->get_hp());
        status_bar_2P.set_mp(player_2->get_mp());

        if (is_game_over) { // 如果游戏结束
            // 累加结算标题两层图片的水平位置
            pos_img_winner_bar.x += (int)(speed_winner_bar * delta);
            pos_img_winner_text.x += (int)(speed_winner_text * delta);
            // 根据是否划出执行不同指令
            if (!is_slide_out_started) // 如果没有执行划出逻辑，说明正在进行停顿展示
            {
                timer_winner_slide_in.on_update(delta);
                // 更新逻辑不让图片超过最大展示位置
                if (pos_img_winner_bar.x > pos_x_img_winner_bar_dst)
pos_img_winner_bar.x = pos_x_img_winner_bar_dst;
                if (pos_img_winner_text.x > pos_x_img_winner_text_dst)
pos_img_winner_text.x = pos_x_img_winner_text_dst;
            }
            else timer_winner_slide_out.on_update(delta);

        }
    }
```

4.7.2 结算动画

在 GameScene 类的 on_draw 函数中，绘制结算动画。

```
    void on_draw(const Camera& camera) {
        putimage_alpha(camera, pos_img_sky.x, pos_img_sky.y, &img_sky);
        putimage_alpha(camera, pos_img_hills.x, pos_img_hills.y, &img_hills);

        for (const auto& platform : platform_list) platform.on_draw(camera); // 通过迭代
遍历平台列表

        if (is_debug) {
            settextcolor(RGB(255, 0, 0));
            outtextxy(15, 15, _T("已开启调试，Q键关闭"));
        }

        player_1->on_draw(camera);
        player_2->on_draw(camera);

        for (const Bullet* bullet : bullet_list) bullet->on_draw(camera); // 按帧绘制

        if (is_game_over) { // 游戏结束绘制黑幕和文本
            putimage_alpha(pos_img_winner_bar.x, pos_img_winner_bar.y,
&img_winner_bar);
            putimage_alpha(pos_img_winner_text.x, pos_img_winner_text.y, player_1-
>get_hp() > 0 ? &img_1P_winner : &img_2P_winner); // 绘制胜利者文本
        }else {
            // 绘制状态条长度
```

```
            status_bar_1P.on_draw();
            status_bar_2P.on_draw();
        }

    }
```

五、测试与优化

5.1 测试环境

游戏在以下环境中进行测试：

操作系统 ：Windows 10

处理器 ：Intel Core i5

内存 ：8GB

显卡 ：NVIDIA GeForce GTX 1050

操作系统 ：Windows 11

处理器 ：Intel Core i9

内存 ：32GB

显卡 ：NVIDIA GeForce GTX 4060

5.2 测试内容

游戏进行了以下测试：

功能测试 ：测试游戏的各项功能是否正常，包括角色选择、场景切换、角色控制、碰撞检测、动画效果、音效处理、游戏结算等。

性能测试 ：测试游戏的帧率、响应时间等性能指标，确保游戏画面流畅，玩家操作及时响应。

兼容性测试 ：测试游戏在不同分辨率、不同硬件配置下的兼容性，确保游戏在各种环境下都能正常运行。

5.3 优化措施

游戏进行了以下优化：

代码优化 ：优化游戏代码，减少冗余代码，提高代码效率。

资源优化 ：优化游戏资源，减少资源占用，提高游戏性能。

算法优化：优化游戏算法，提高算法效率，减少计算量。

六、总结与展望

6.1 项目总结

通过本次课程设计，成功实现了一个 2D 格斗游戏，游戏具有完整的功能和良好的用户体验。在项目开发过程中，深入理解和掌握了 C++ 编程语言在游戏开发中的应用，包括面向对象编程、图形渲染、碰撞检测、动画处理等核心技术，同时培养了逻辑思维、问题解决能力。

6.2 未来展望

在未来的学习和实践中，将继续深入研究游戏开发技术，探索更复杂的游戏机制和算法，尝试开发更具挑战性和创新性的游戏项目，不断提升技术水平和实践能力。